



Project Report

Smart Spending, Student Style - a budget and expense tracker for students

End-to-End Web Solutions · Theme: NextGen BudgetBee

Student	Hamza Haroon
Centre	Aptech Metro Star Gate, Karachi
Stack	MongoDB, Express.js, React, Node.js (MERN)
Live site	https://campuscoin-sable.vercel.app
Source code	https://github.com/hamzaharoonk07-cloud/campuscoin
Version	1.0 · September 2026

Contents

1	Problem Definition
2	Proposed Solution and Scope
3	Design Specifications
4	Diagrams: Architecture, Flowcharts and Data Flow Diagrams
5	Database Design
6	Functional Requirements and How They Are Met
7	Non-Functional Requirements
8	User Interface
9	Test Data and Testing
10	Project Installation Instructions
11	User Credentials
12	Acknowledgement of AI Tools
13	Conclusion and Future Work

1. Problem Definition

College and university students receive money from several irregular sources: a monthly allowance from home, part-time or tutoring work, scholarship instalments and occasional gifts such as Eidi. Very few of them know where that money goes by the end of the month. The personal-finance apps that exist are designed for salaried adults with a fixed pay day and a bank feed; they are often subscription-based, ask for bank details, and use categories that do not match a student's life.

A student's spending is made of small, frequent purchases – chai at the canteen, a rickshaw to campus, photocopies and printing, a shared Netflix plan – together with a few large, fixed costs such as hostel rent. These small items are easy to forget, and they are exactly the ones that add up. Without a simple record, a student usually discovers a problem only when the money has already run out.

1.1 The problem in short

- Students have no quick, free way to record income and expenses in categories that fit student life.
- They cannot see which category is growing, or compare this month with their own usual spending.
- Generic advice ("spend less") is not useful; advice needs to come from the student's own numbers.
- Linking a bank account is neither possible nor desirable for most students.

1.2 Objectives

- Let a student record a transaction in a few seconds, with the category suggested automatically.
- Show the month at a glance: balance, top category, budget against actual spending.
- Set a budget per category and warn the student in time, at 80% and at 100%.
- Produce monthly reports, a plain-language monthly insight and ranked saving tips from the student's own history.
- Give an administrator control over default categories, announcements and user accounts.
- Work on phones and computers, in light and dark mode, without any bank connection.

2. Proposed Solution and Scope

Campus Coin is a responsive full-stack web application. A student registers, logs in and immediately starts recording income (allowance, part-time job, scholarship, gift, other income) and expenses (food, transport, hostel/rent, academics, subscriptions, entertainment, miscellaneous). Every transaction is stored in a central MongoDB database, which powers the dashboard, category-wise and six-month reports, the calendar, budgets and alerts, the saving-tips engine and the monthly insights.

An intelligent assistant runs through the application: it suggests a category while the student types a description and learns from corrections, reads the amount, shop and date from a photographed receipt, writes a monthly summary that flags unusual growth, and answers questions in a chat window ("How much did I spend on food?") from the student's own data.

2.1 Scope

In scope	Out of scope (by the SRS constraints)
Manual entry, CSV import and receipt scanning of transactions	Linking to real bank accounts or wallets
Budgets, alerts, reports, insights, saving tips, calendar, chat assistant	Real payments or money transfers
Student and administrator accounts with secure sessions and email	Certified financial advice (all advice is marked as a suggestion)
Responsive layout, dark mode, text-size control	Native mobile apps (the web app works on phones instead)

2.2 Constraints followed

- No bank integration: all data is entered by the student, imported from a CSV file or read from a receipt photo.
- Every suggestion from the categoriser, insight writer and tips engine can be reviewed and overridden, and is labelled as advice, not a decision.
- No ready-made website template or UI component library was used. The design system, layout and every chart are written in CSS and SVG as part of the project.

3. Design Specifications

3.1 Technology

Layer	Technology	Why it was chosen
Front end	React 18, React Router, Vite	Component-based screens, fast builds, one-page navigation without reloads
Styling and charts	Hand-written CSS design tokens; SVG charts	Full control of the design; charts that stay sharp on any screen; no template
Back end	Node.js, Express 5	One language (JavaScript) across the stack; simple REST routes
Database	MongoDB with Mongoose	Flexible documents for transactions, insights and recurring rules; free Atlas hosting
Security	bcrypt, JSON Web Tokens, crypto tokens	Salted password hashes, signed sessions, one-time reset links
Receipt reading	Tesseract.js (in the browser)	OCR without uploading the photo or paying for an API
Email	Nodemailer with Gmail SMTP	Password links, welcome and summary emails
Optional AI wording	Anthropic Claude API	Rewrites already-computed monthly figures in warmer language
Hosting	Vercel (serverless API + static client), MongoDB Atlas	Free, always-on public deployment with HTTPS

3.2 Modules

Module	Responsibility
Authentication	Registration, student and separate admin sign-in, logout after 5 wrong passwords, sessions that end on password change, "sign out everywhere", password recovery and change by emailed link
Profile and settings	Name, study level (Matric to PhD), institution, monthly allowance, savings goal, currency, photo, theme and text size
Categories	Five default income and seven default expense categories; students add, edit and delete their own ("Manage own categories")
Transactions	Quick add, recurring entries, edit and delete, search and filters, CSV import with batch suggestions, CSV export, receipt photos
Categoriser (AI)	Suggests a category from the description using the student's own word-to-category history; learns from every correction

Module	Responsibility
Anomaly detection	Flags unusually large amounts and likely duplicates
Budgets and alerts	A monthly cap per category, live progress bars, in-app notifications at 80% and 100%, copy to next month
Dashboard and calendar	Greeting, balance, top category, budget vs actual, cash flow, spending donut; a calendar of every day's money
Reports	Category-wise month, six-month income vs expense, daily and weekly summaries, filters, forecast, PDF and image export
Insights	Monthly narrative, month score, category changes, spending habits, history, bookmarks, email sharing
Saving tips	Rules that compare the month with the student's own averages, ranked by money saved; pin and dismiss
Chat assistant (Coin)	Answers money questions from the same figures the reports use
Administration	Usage statistics, student accounts (view, disable, reset, remove), default categories, announcements and tip templates

3.3 Security design

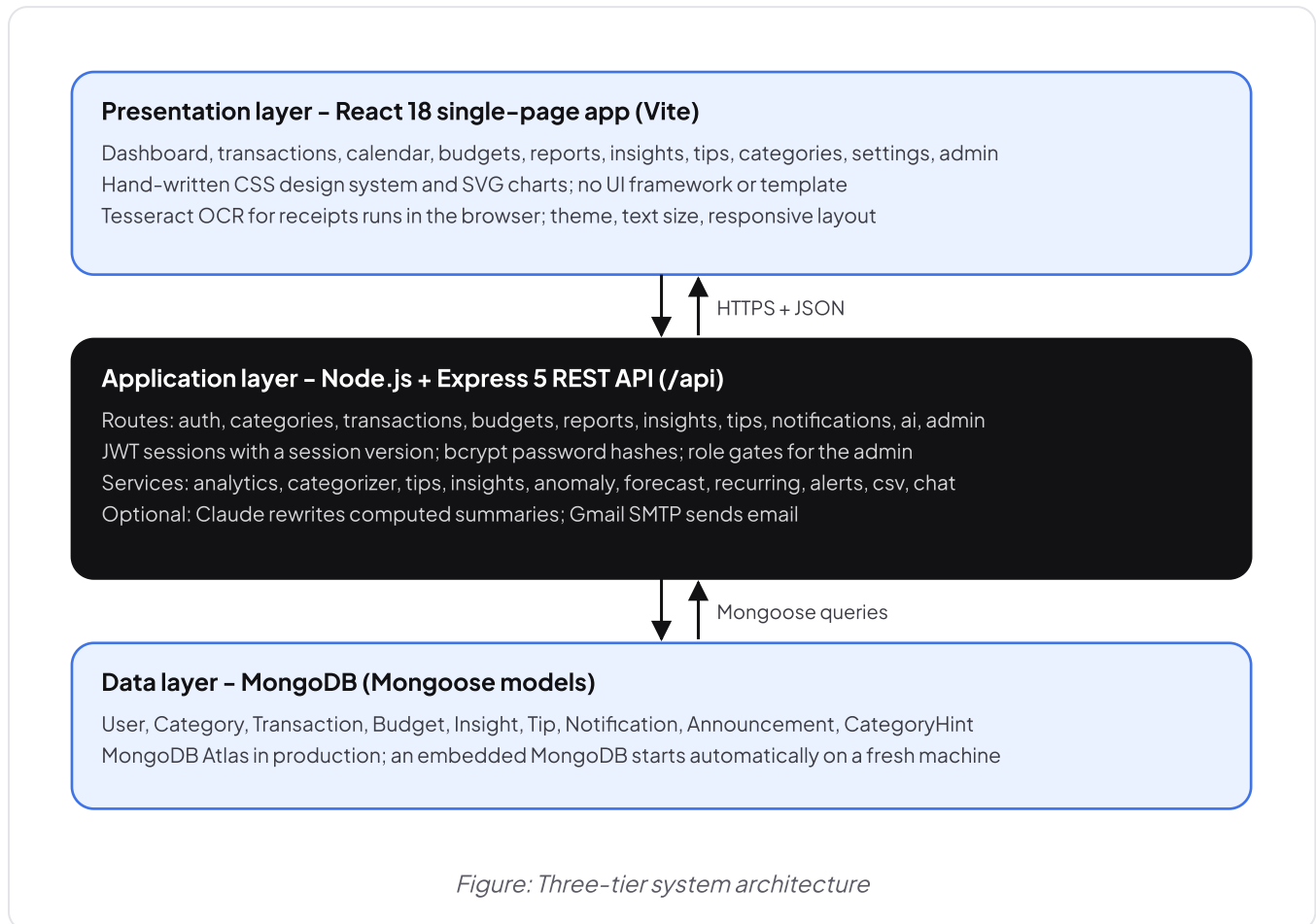
- Passwords are stored only as salted bcrypt hashes (cost 10) and must be at least 8 characters with letters and numbers, not a common password and not built from the user's name or email.
- Each session token carries the account's session version; changing or resetting a password raises it, which ends every other session immediately.
- Five wrong passwords in a row lock sign-in for 15 minutes. Disabled accounts lose access at once.
- A password can only change through a one-time link, valid for one hour, emailed to the account's own address. Only the SHA-256 hash of the link's token is stored.
- Every data query is filtered by the signed-in user, so a student can only ever read their own transactions. Admin routes check the admin role on every request.

3.4 User-interface design

The interface follows a black, white and blue scheme: a slim icon sidebar that opens on hover, one white content panel, black pill buttons and a single blue (#5B91FF) for figures and money coming in. Text is set in Plus Jakarta Sans. Categories use full-colour pictures, and merchants such as Netflix, Spotify and foodpanda show their own logos. On phones the sidebar becomes a bottom tab bar and cards stack in one column. A light and dark theme, adjustable text size and breadcrumbs are provided for accessibility.

4. Diagrams

4.1 System architecture



The application follows the three-tier design described in the SRS. The React client never talks to the database directly; it calls the Express API over HTTPS, and every figure shown on screen is calculated by the server's service layer from the database.

4.2 Flowcharts

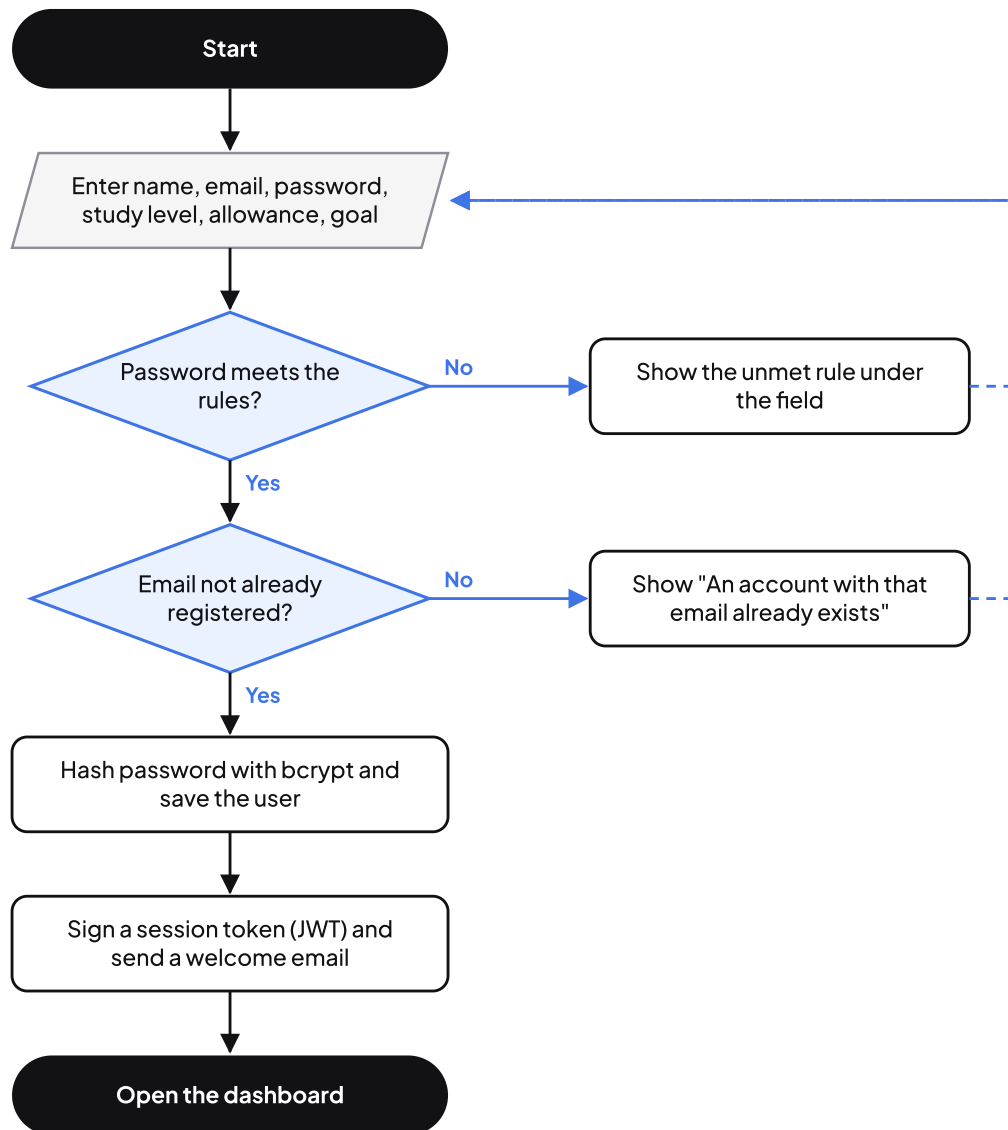


Figure: Flowchart – student registration

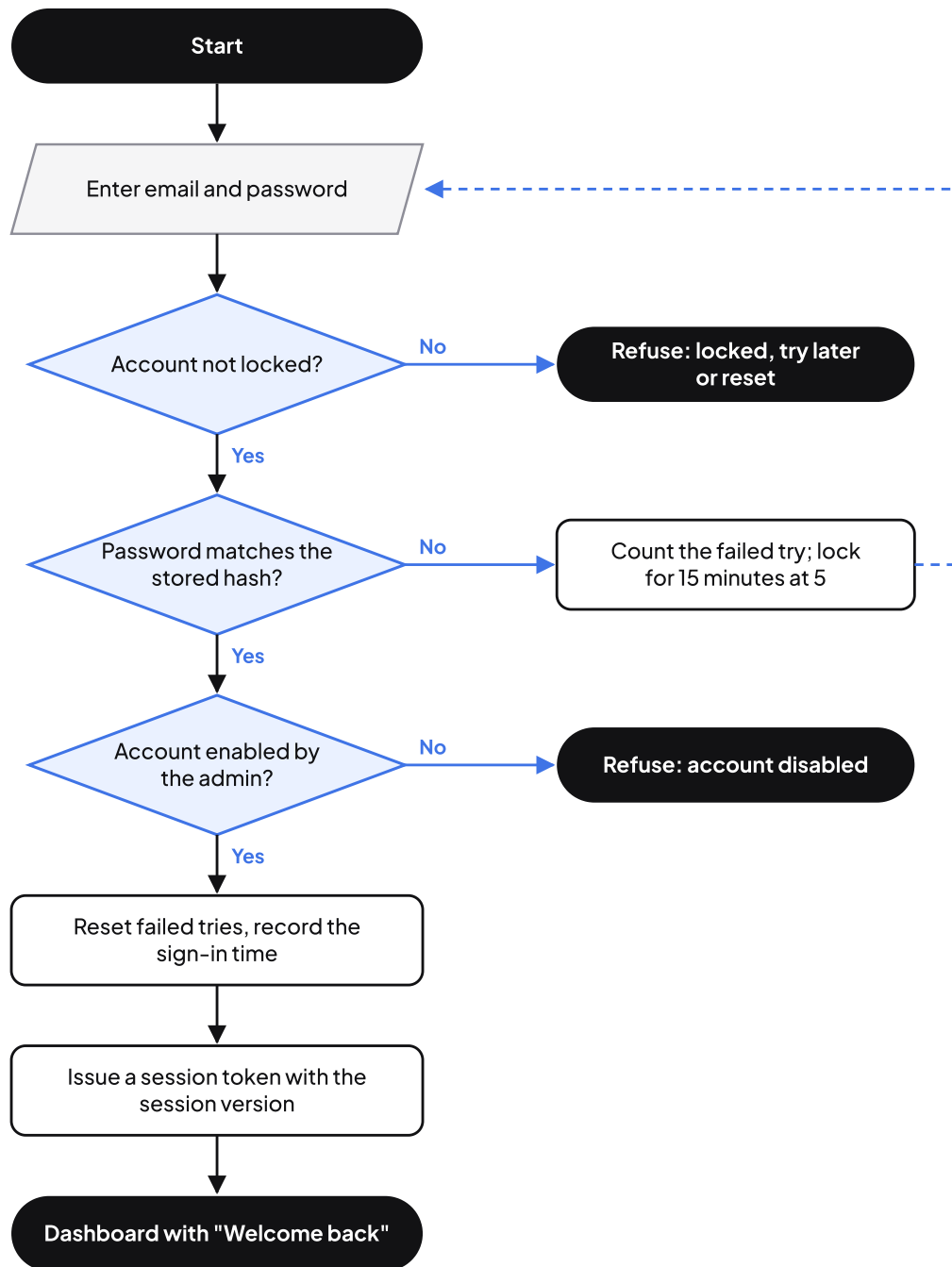


Figure: Flowchart - student sign-in with lockout

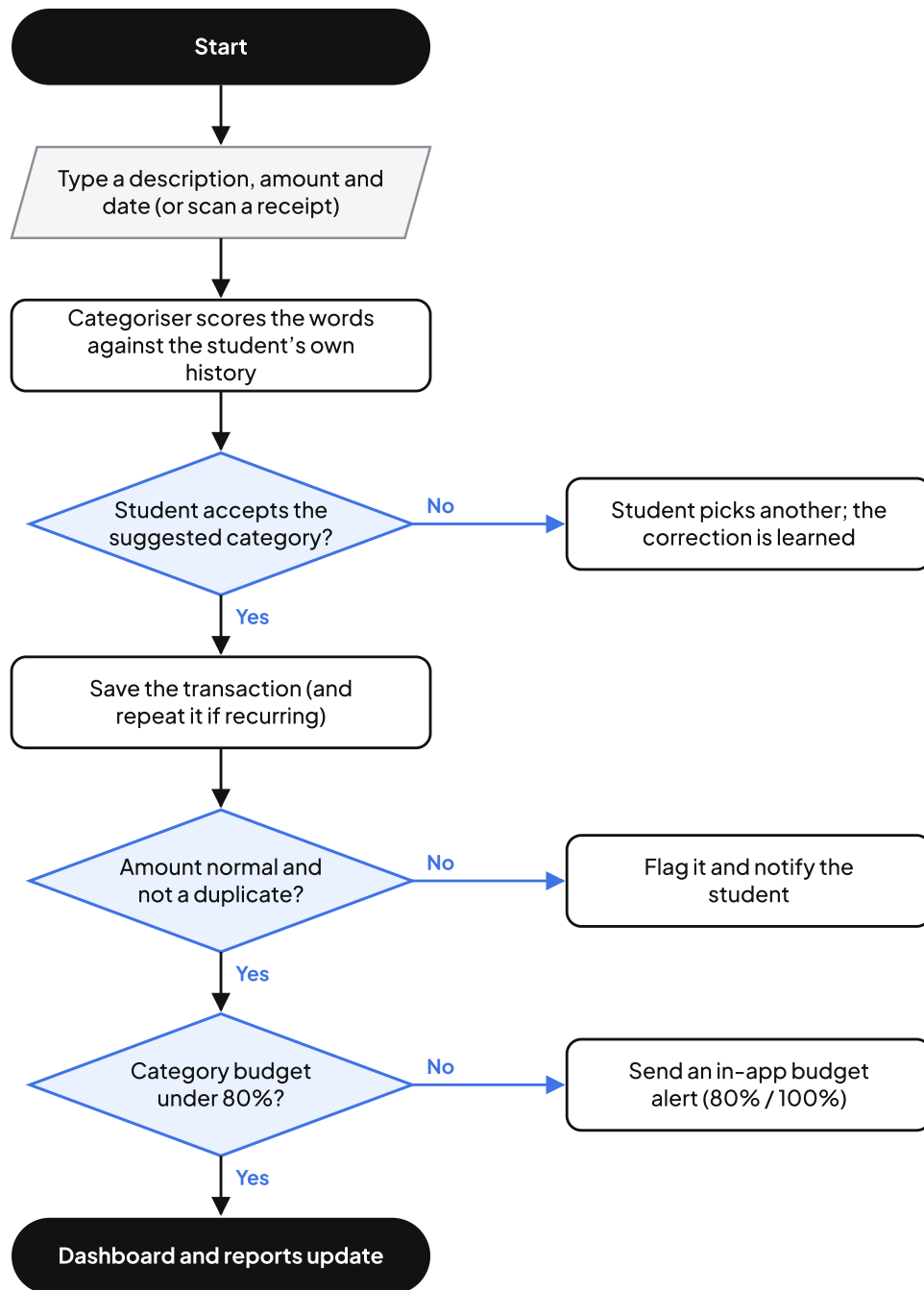


Figure: Flowchart - adding a transaction with AI categorisation

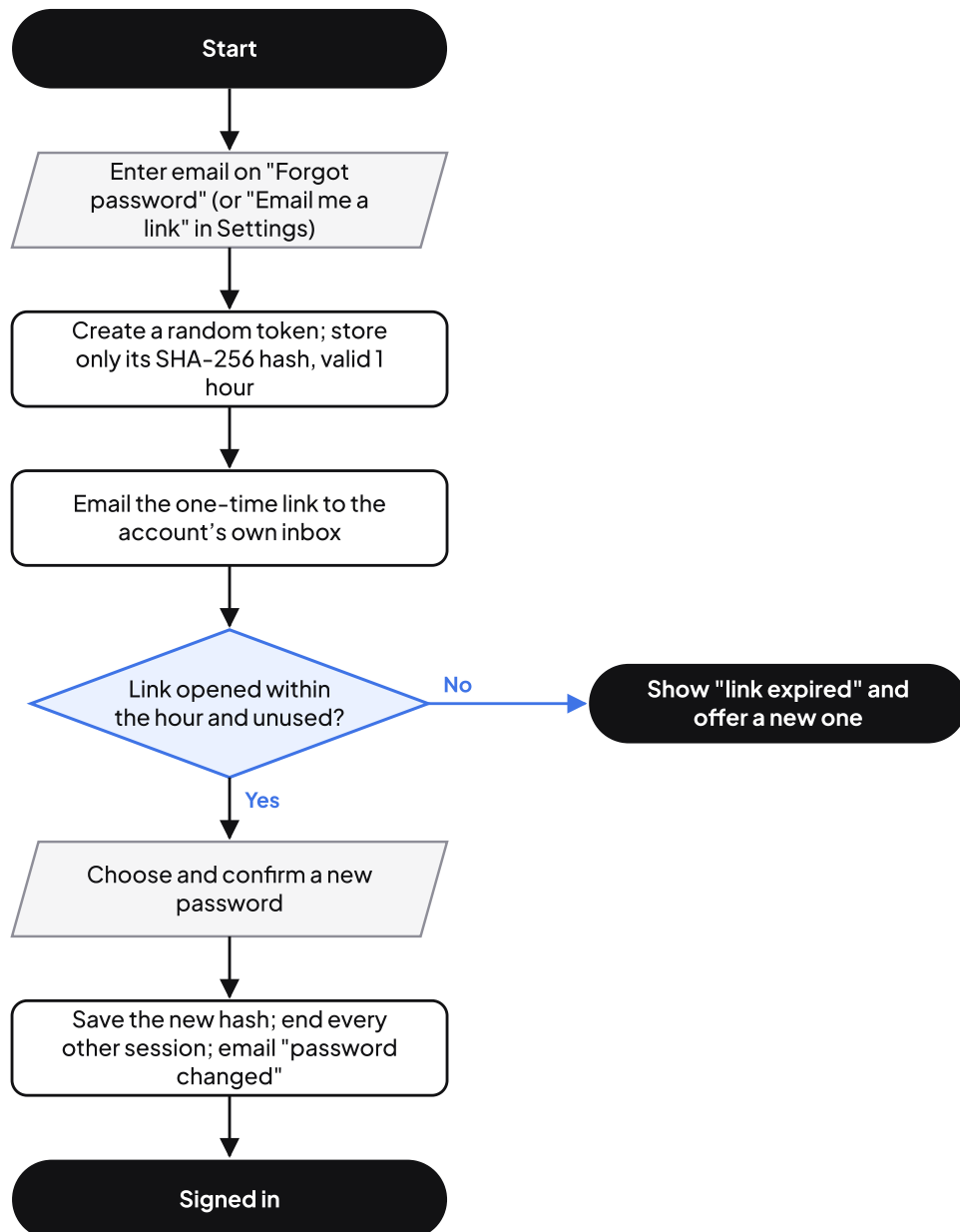


Figure: Flowchart - password recovery and change by tokenised link

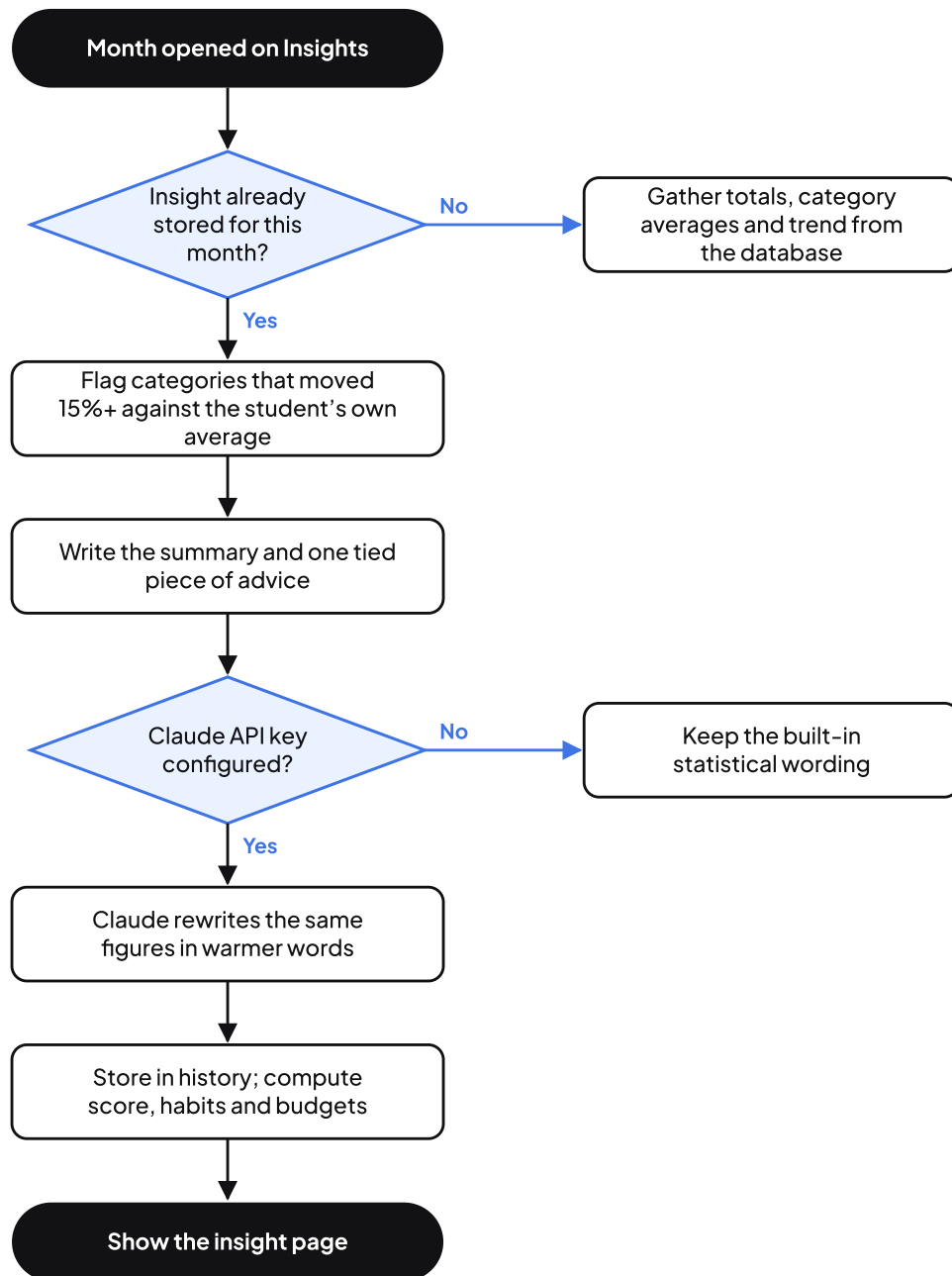


Figure: Flowchart - generating the monthly insight

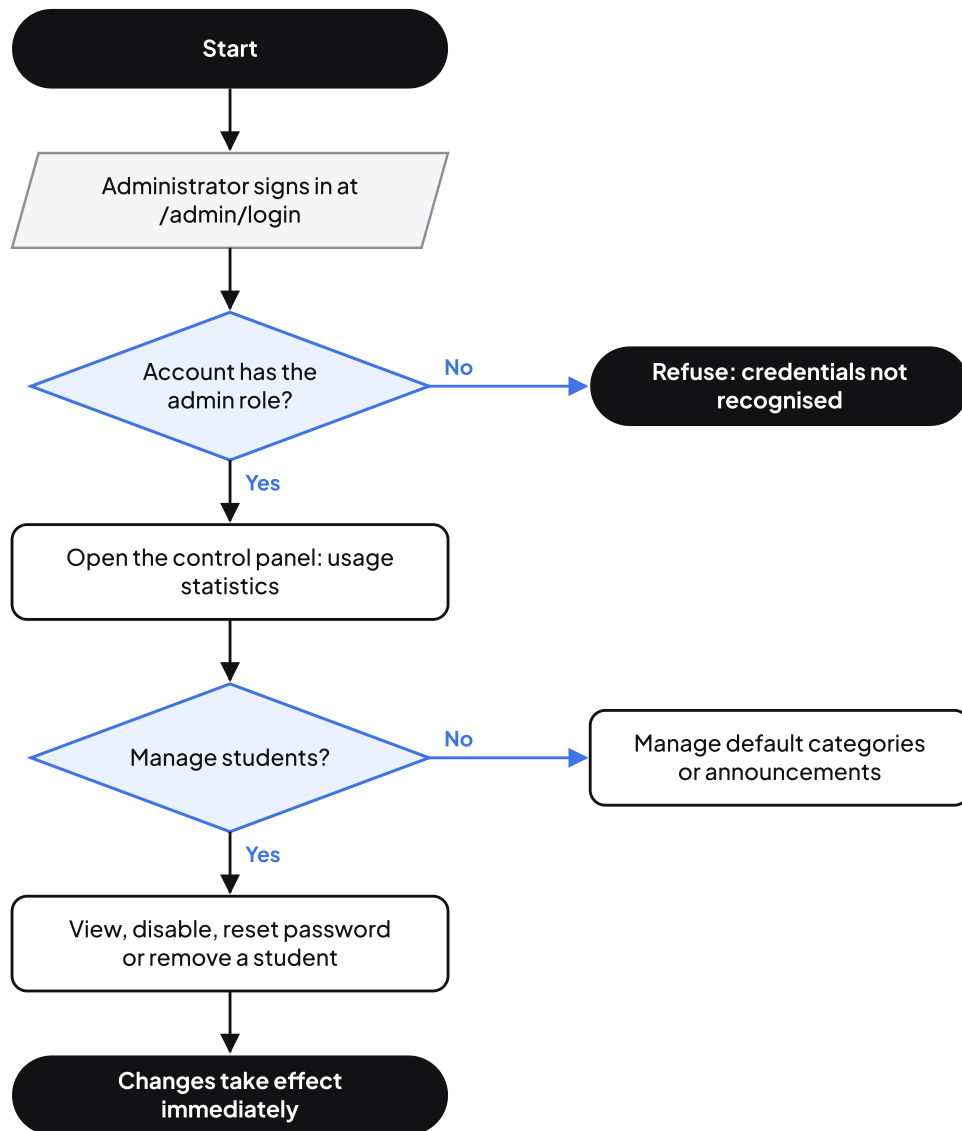
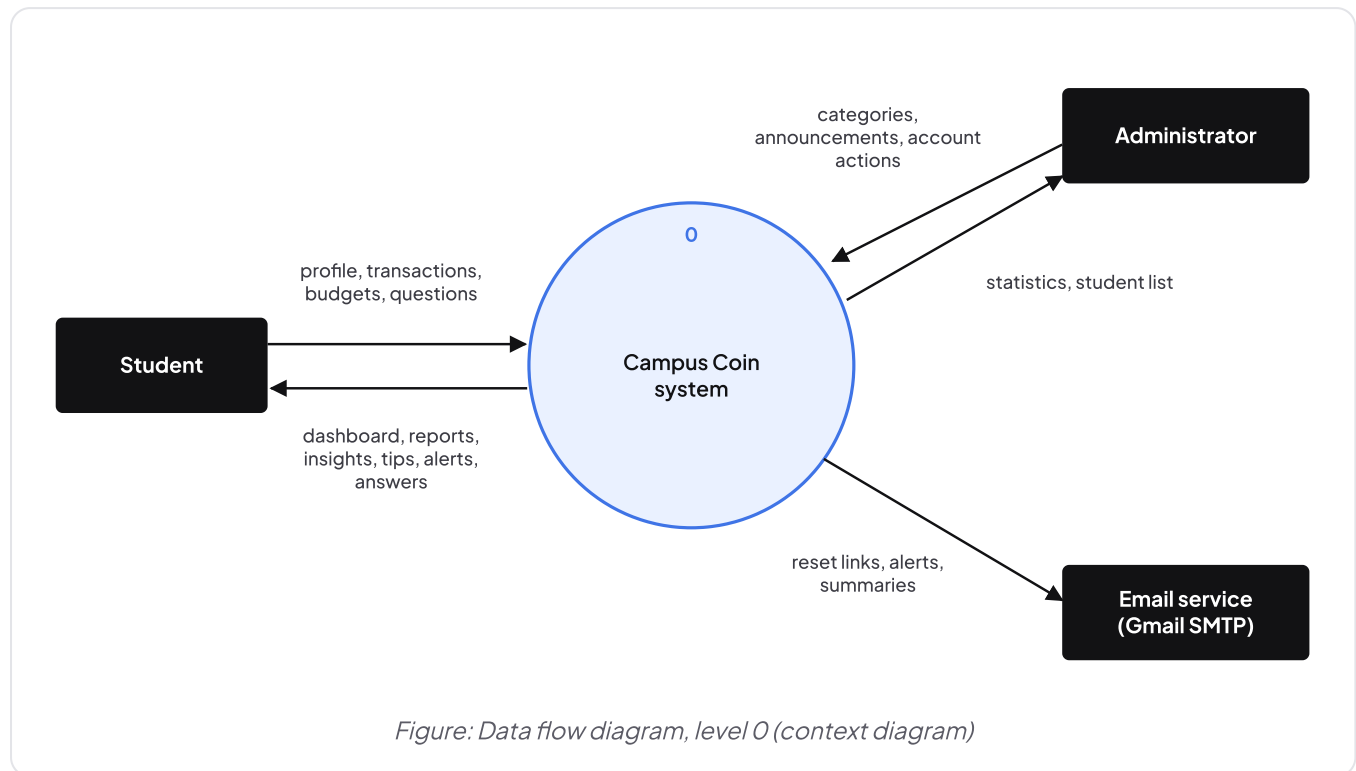


Figure: Flowchart - administrator control panel

4.3 Data flow diagrams



At the context level, the student sends profile details, transactions, budgets and questions into the system and receives the dashboard, reports, insights, tips, alerts and answers. The administrator manages categories, announcements and accounts and receives usage statistics. The email service delivers password links, alerts and summaries.

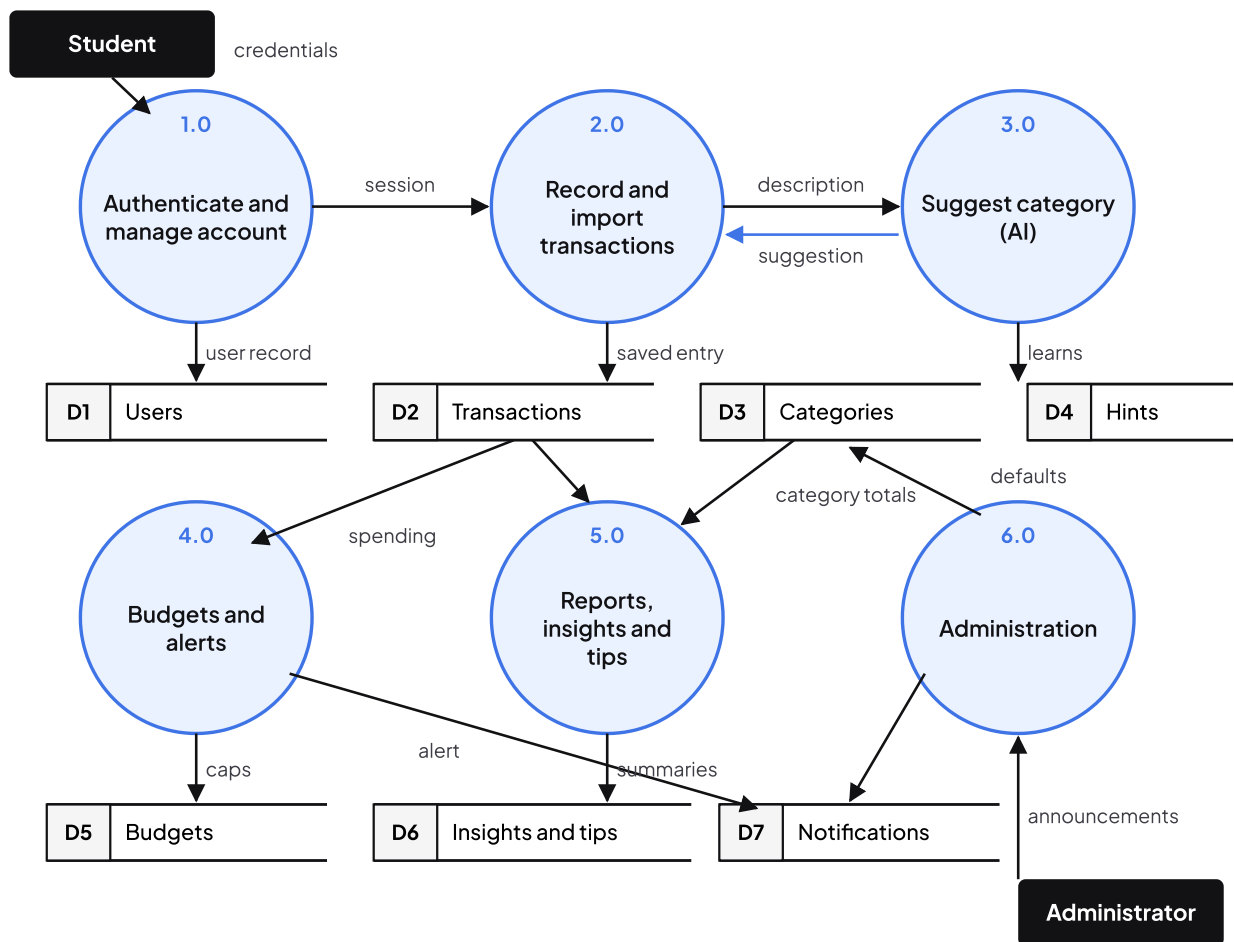
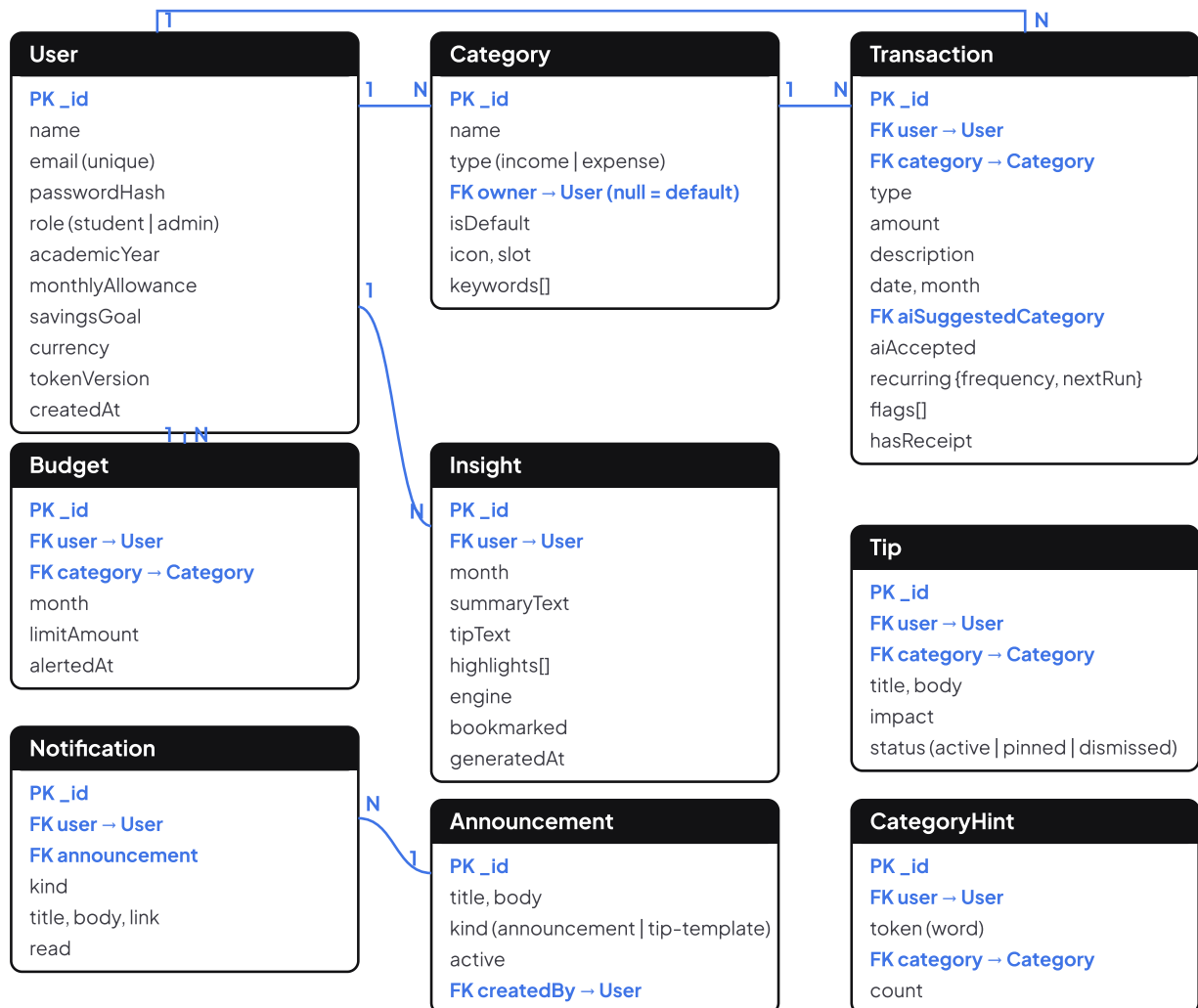


Figure: Data flow diagram, level 1

At level 1, the system splits into six processes that share seven data stores. Recording a transaction (2.0) asks the categoriser (3.0) for a suggestion, which learns from the saved result (D4). Budgets (4.0) read spending from the transactions store and write alerts to the notifications store. Reports, insights and tips (5.0) read transactions and categories and store insight and tip history (D6). Administration (6.0) maintains default categories and pushes announcements to every student's notifications.

5. Database Design

Campus Coin uses MongoDB, a document database. Each entity below is a collection; relationships are stored as references (ObjectId) to the related document, which is the MongoDB equivalent of a foreign key. Every collection also records `createdAt` and `updatedAt` automatically.



Every collection except Category defaults and Announcement belongs to one User (1 : N); see the relationship table below.

Figure: Entity-relationship diagram (MongoDB collections)

5.1 Relationships

Relationship	Type	Meaning
User → Transaction	1 : N	A student owns many transactions
Category → Transaction	1 : N	Each transaction belongs to one category
User → Category	1 : N	A student's own categories (owner); defaults have no owner
User → Budget, Category → Budget	1 : N	One cap per category per month

Relationship	Type	Meaning
User → Insight	1 : N	One insight per month, kept as history
User → Tip, Notification, CategoryHint	1 : N	Personal tips, alerts and learned word pairings
Announcement → Notification	1 : N	An announcement is copied into each student's bell

5.2 Collections

User

Field	Type	Key	Description
<code>_id</code>	ObjectId	PK	Unique identifier
<code>name</code>	String		Full name
<code>email</code>	String	Unique	Login email
<code>passwordHash</code>	String		bcrypt hash of the password (never returned to the client)
<code>role</code>	String		student or admin
<code>academicYear</code>	String		Study level: Matric, O/A Levels, Intermediate, university year, Masters, MPhil, PhD
<code>institution</code>	String		School, college or university
<code>monthlyAllowance</code>	Number		Monthly allowance baseline
<code>savingsGoal</code>	Number		Monthly savings goal
<code>currency</code>	String		PKR, USD, EUR, GBP, INR or AED
<code>avatar, avatarColor</code>	String		Profile photo (small data URL) or initial colour
<code>preferences</code>	Object		theme, fontScale, reducedMotion, alertsEnabled
<code>disabled</code>	Boolean		Set by the administrator to block sign-in
<code>tokenVersion</code>	Number		Session version; raised to end all sessions
<code>failedLogins, lockUntil</code>	Number, Date		Sign-in lockout
<code>resetTokenHash, resetTokenExpires</code>	String, Date		One-time password link (hash only)
<code>lastLoginAt, passwordChangedAt</code>	Date		Account history

Category

Field	Type	Key	Description
<code>_id</code>	ObjectId	PK	Unique identifier
<code>name</code>	String		e.g. Food, Transport, Hostel/Rent
<code>type</code>	String		income or expense

Field	Type	Key	Description
owner	ObjectId	FK → User	Empty for a system default
isDefault	Boolean		System default or student-created
icon, slot	String, Number		Picture and colour
keywords	String[]		Seed words for the categoriser
archived	Boolean		Hidden but kept for old transactions

Transaction

Field	Type	Key	Description
_id	ObjectId	PK	Unique identifier
user	ObjectId	FK → User	Owner
category	ObjectId	FK → Category	Chosen category
type	String		income or expense
amount	Number		Amount (positive)
description, note	String		Free text entered by the student
date, month	Date		Date of the transaction and its month (for fast monthly queries)
aiSuggestedCategory	ObjectId	FK → Category	Category the assistant suggested
aiAccepted	Boolean		Whether the suggestion was kept (accuracy tracking)
recurring	Object		enabled, frequency (weekly/monthly), nextRun
flags	String[]		large, duplicate
source	String		manual, csv, recurring or receipt
receipt, hasReceipt	String, Boolean		Compressed receipt photo

Budget

Field	Type	Key	Description
_id	ObjectId	PK	Unique identifier
user	ObjectId	FK → User	Owner
category	ObjectId	FK → Category	Capped category
month	Date		Month the budget applies to
limitAmount	Number		The cap
alertedAt	Number		Last alert level sent (0, 80 or 100)

Insight

Field	Type	Key	Description
<code>_id</code>	ObjectId	PK	Unique identifier
<code>user</code>	ObjectId	FK → User	Owner
<code>month</code>	Date		Month covered
<code>summaryText</code>	String		Narrative summary
<code>tipText</code>	String		Actionable advice
<code>highlights</code>	Array		Categories that moved, with change %
<code>stats</code>	Object		income, expense, balance, savings rate, top category
<code>engine</code>	String		statistical or IIm
<code>bookmarked</code>	Boolean		Saved by the student
<code>generatedAt</code>	Date		Time of generation

Tip, Notification, Announcement and CategoryHint

Collection	Main fields	Purpose
Tip	user, key, title, body, impact, category, status (active/pinned/dismissed)	Saving tips ranked by impact
Notification	user, kind, title, body, link, read, announcement	Budget alerts, anomaly checks, news in the bell
Announcement	title, body, kind (announcement/tip-template), active, createdBy	System-wide notices and tip templates
CategoryHint	user, token (word), category, count	What the categoriser has learned for each student

6. Functional Requirements and How They Are Met

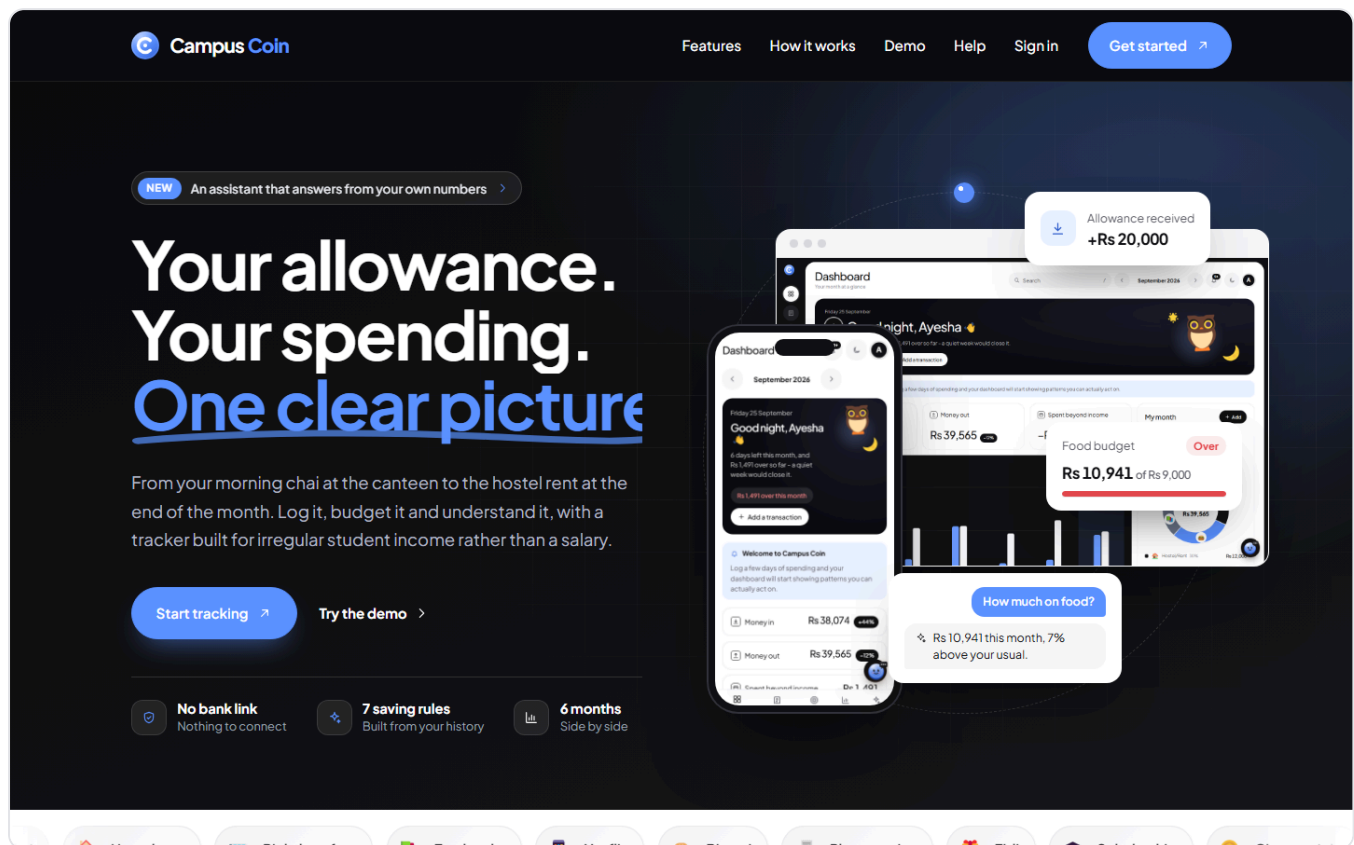
SRS requirement	Implementation in Campus Coin
Student registration and login; separate admin login	/register and /login for students; /admin/login refuses any non-admin account
Secure session management	Signed JWT with session version, logout, "sign out everywhere", immediate block for disabled accounts
Password recovery by tokenised link	One-time 1-hour link emailed through Gmail SMTP; password change also works only by link
Editable profile: name, academic year, allowance, savings goal	Account settings, plus institution, currency, photo and preferences
Bulk import from CSV	Import wizard with preview and batch category suggestions; CSV export too
Default and personal income/expense categories	All 12 defaults seeded; "Manage own categories" to add, edit, delete
Dashboard: greeting, balance, quick add	Time-of-day greeting card, money in/out/kept, quick-add tiles for chai, rickshaw, printing, allowance
Saving tips and highlights on the dashboard	"Worth doing" and "Coin's insight" cards
Widgets: This Month's Top Category, Budget vs. Actual	Both present as their own cards
Quick-add, recurring entries, edit and delete	Transaction form, weekly/monthly repeats written when due, edit/delete with history kept
AI category suggestion that learns and can be overridden	Per-student word-to-category model; corrections are learned; accuracy tracked
Batch categorisation on CSV import	Every imported row gets a suggested category in the preview
Category-wise, six-month, daily and weekly reports; filters	Reports page with donut, month bars, daily and weekly views, filters by date, category, source
Export report as PDF or image	Report export to PDF and PNG
AI monthly insight, flagged growth, advice, history	Insights page with summary, month score, category changes, habits, stored history
Saving tips ranked by impact; pin or dismiss	Tips page; every tip states its monthly value
Monthly budget per category, live progress, alerts	Budgets page; notifications at 80% and 100%
Bookmark tips and insights; share by email	Pin tips, bookmark insights, email the monthly summary

SRS requirement	Implementation in Campus Coin
Admin: default categories, announcements, users, statistics	Admin control panel with all four areas
Recently viewed, forecast, large/duplicate detection	Recent list, least-squares forecast, anomaly flags
Dark mode, font size, breadcrumbs, loading indicators	Theme toggle, four text sizes, breadcrumbs, loader and shimmering placeholders
AI chatbot	Built-in assistant "Coin", answering from the student's own data
OCR for receipts (optional AI/ML)	Receipt photo read in the browser: amount, shop and date filled in

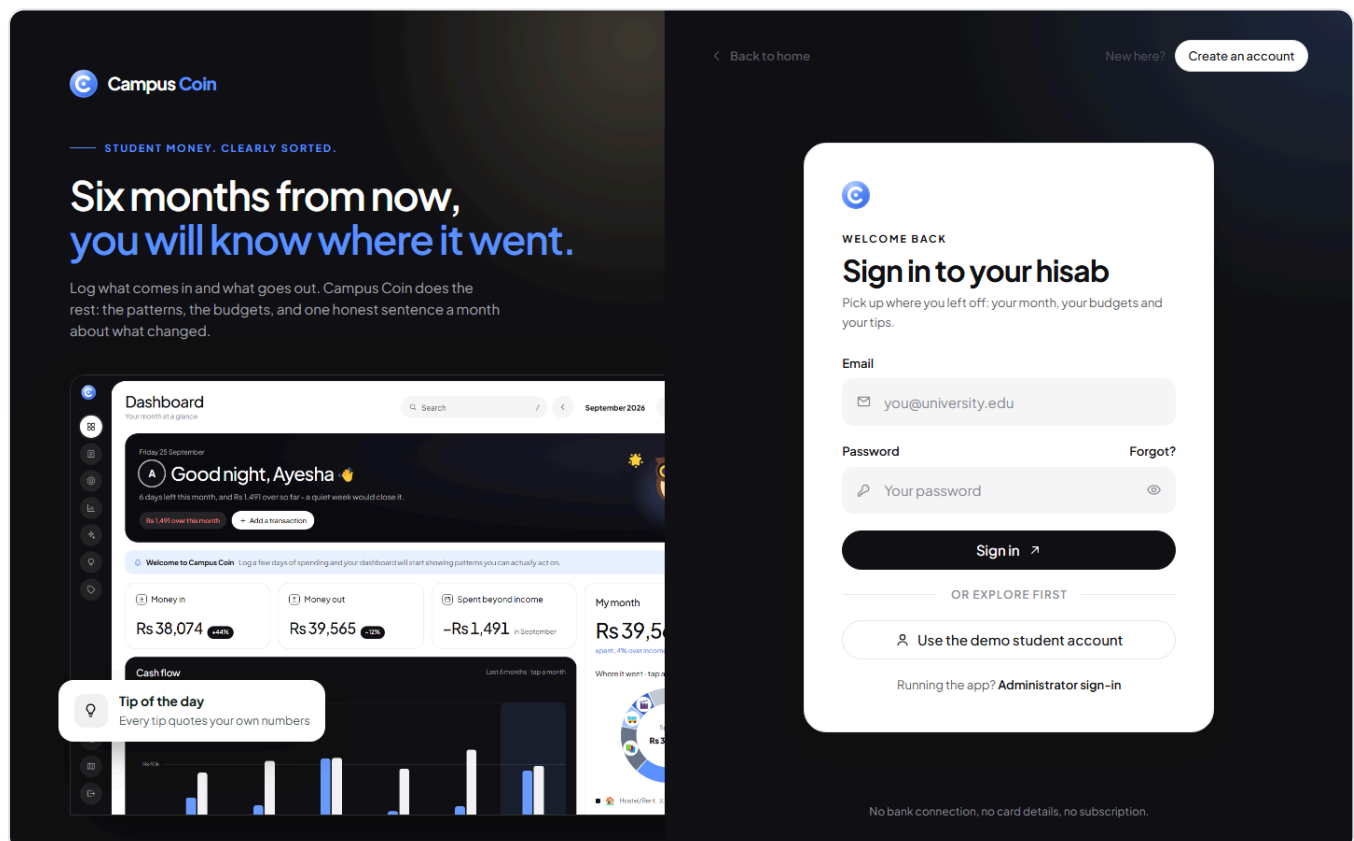
7. Non-Functional Requirements

Requirement	How it is achieved
Safe to use	No downloads except exports the student asks for; uploaded pictures are checked for type and size on the server
Accessibility	Readable font, strong contrast, keyboard focus, labels for screen readers, dark mode, text-size control, reduced-motion support
User-friendliness	A four-step getting-started checklist for new accounts, quick-add buttons, category suggested automatically
Operability and reliability	65 automated end-to-end checks run against the local and the live site
Performance	The dashboard loads in one request; charts are lightweight SVG; indexes on user and month
Scalability	Stateless API on Vercel serverless functions and MongoDB Atlas, both scaling with traffic
Security	Authentication on every data route; each student can read only their own data; hashed passwords; HTTPS
Availability	Hosted on Vercel and MongoDB Atlas, available 24/7
Compatibility	Tested in Chrome and on phone, tablet and laptop widths; works in current browsers

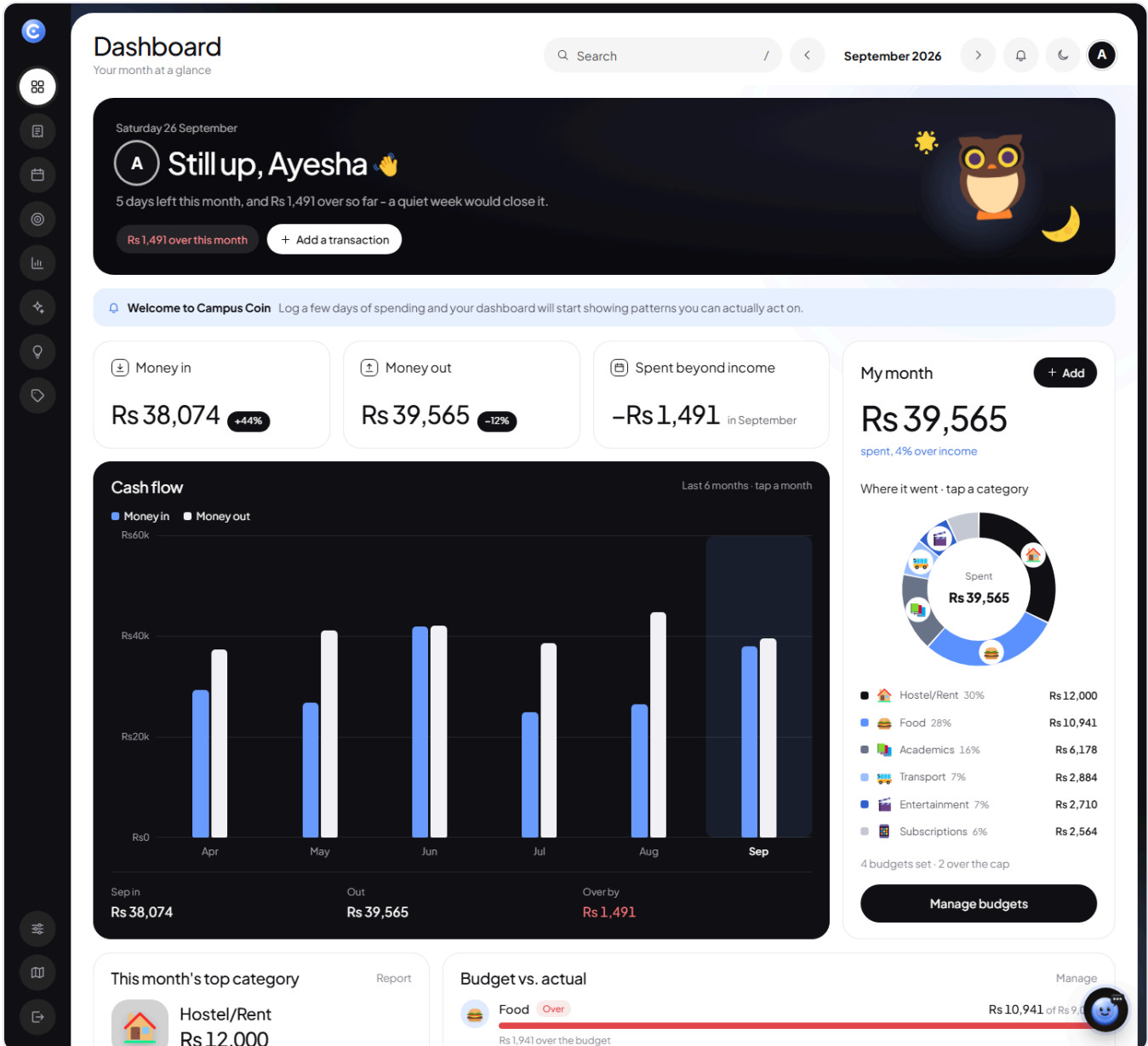
8. User Interface

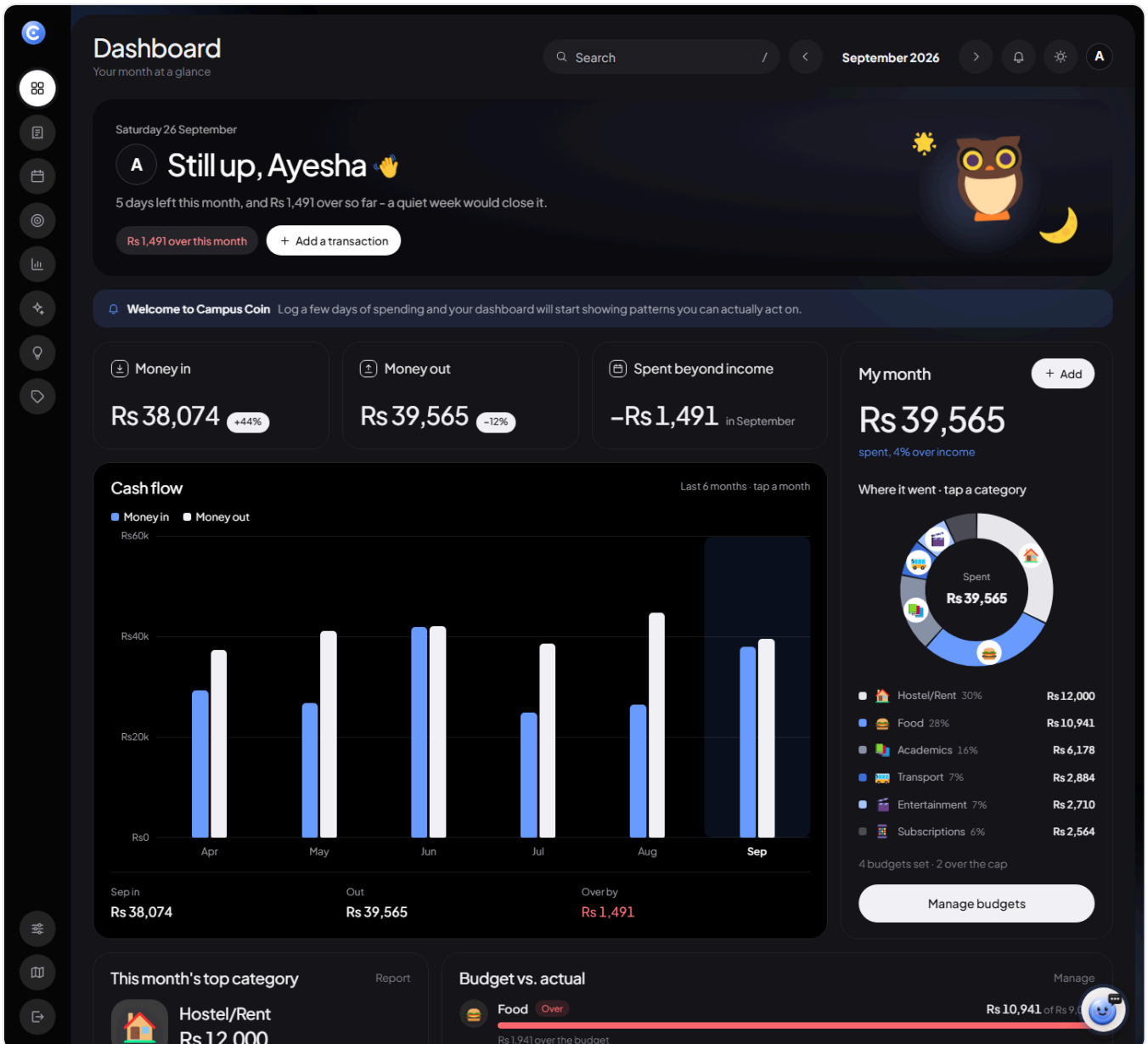


Landing page



Student sign-in with the demo account button





Dashboard in dark mode

Dashboard / Transactions

Transactions

Everything that came in and went out

Search

Direction

In and out

Category

Any category

From

dd/mm/yyyy

To

dd/mm/yyyy

☐ Only ones Campus Coin flagged

257 transactions

Page 1 of 11

Tue 22 Sept

-Rs 2,786

Birthday gift for a friend

Miscellaneous

-Rs 1,051

Printing lab report

Academics

-Rs 1,038

Cricket match with the hostel

Entertainment

-Rs 697

Mon 21 Sept

+Rs 3,074

Birthday money

Gift

+Rs 3,074

Sun 20 Sept

-Rs 1,707

Laundry

Miscellaneous

-Rs 561

Haircut

Miscellaneous

-Rs 676

Biryani with friends

Food

-Rs 470

Sat 19 Sept

-Rs 3,737

Data structures textbook

Academics

-Rs 1,333

Groceries for the room

Food

-Rs 1,257

Campus cafe lunch

Food

-Rs 1,147

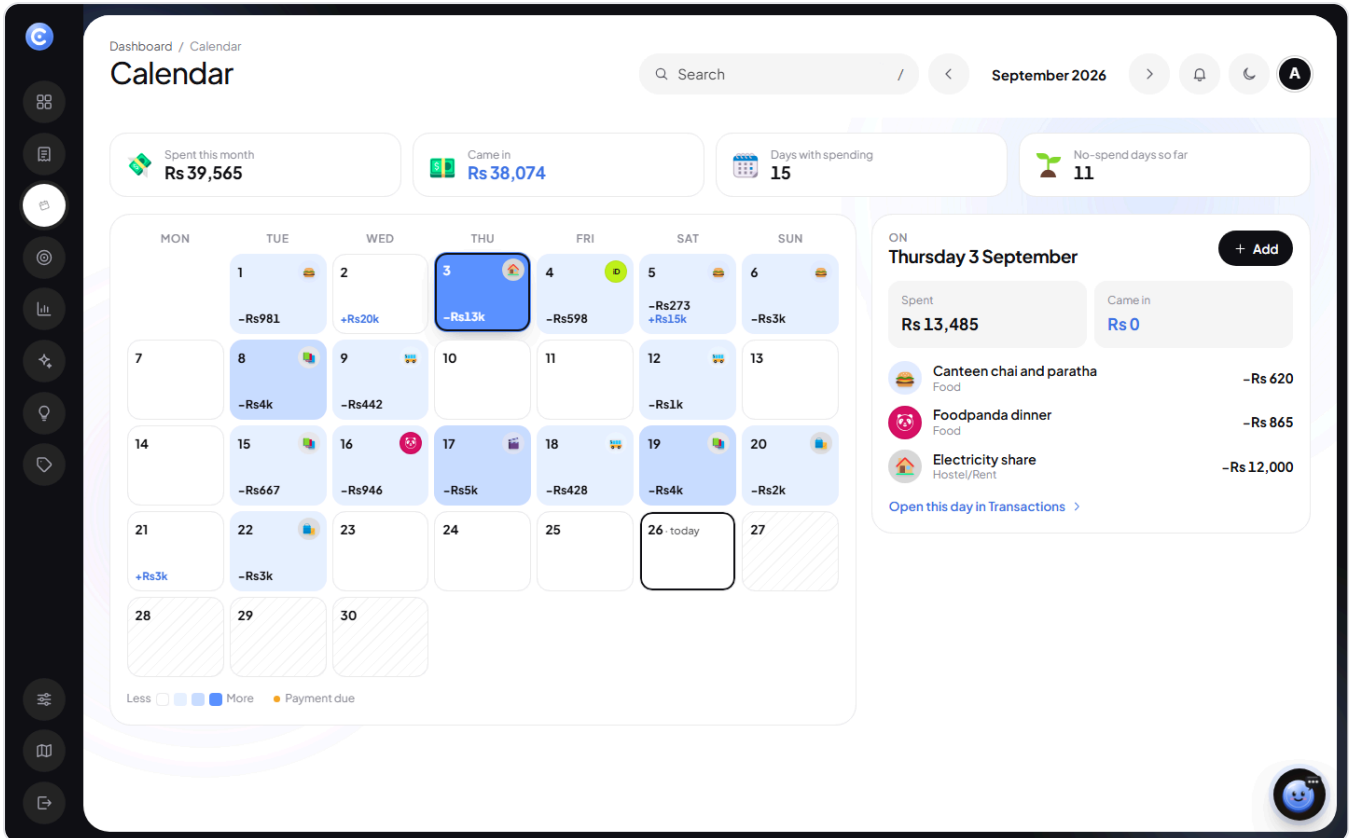
Fri 18 Sept

-Rs 428

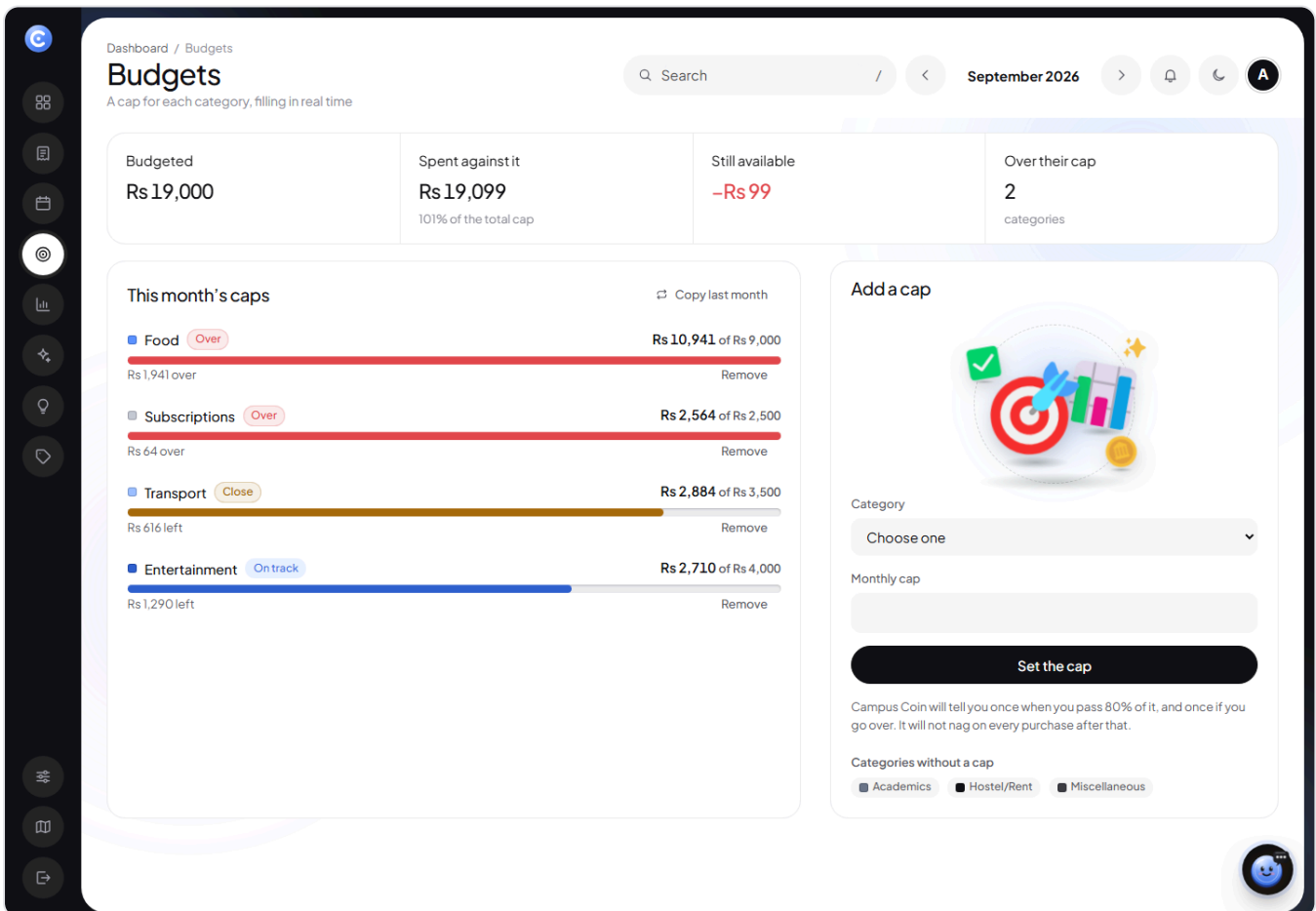
Petrol for the bike

Transport

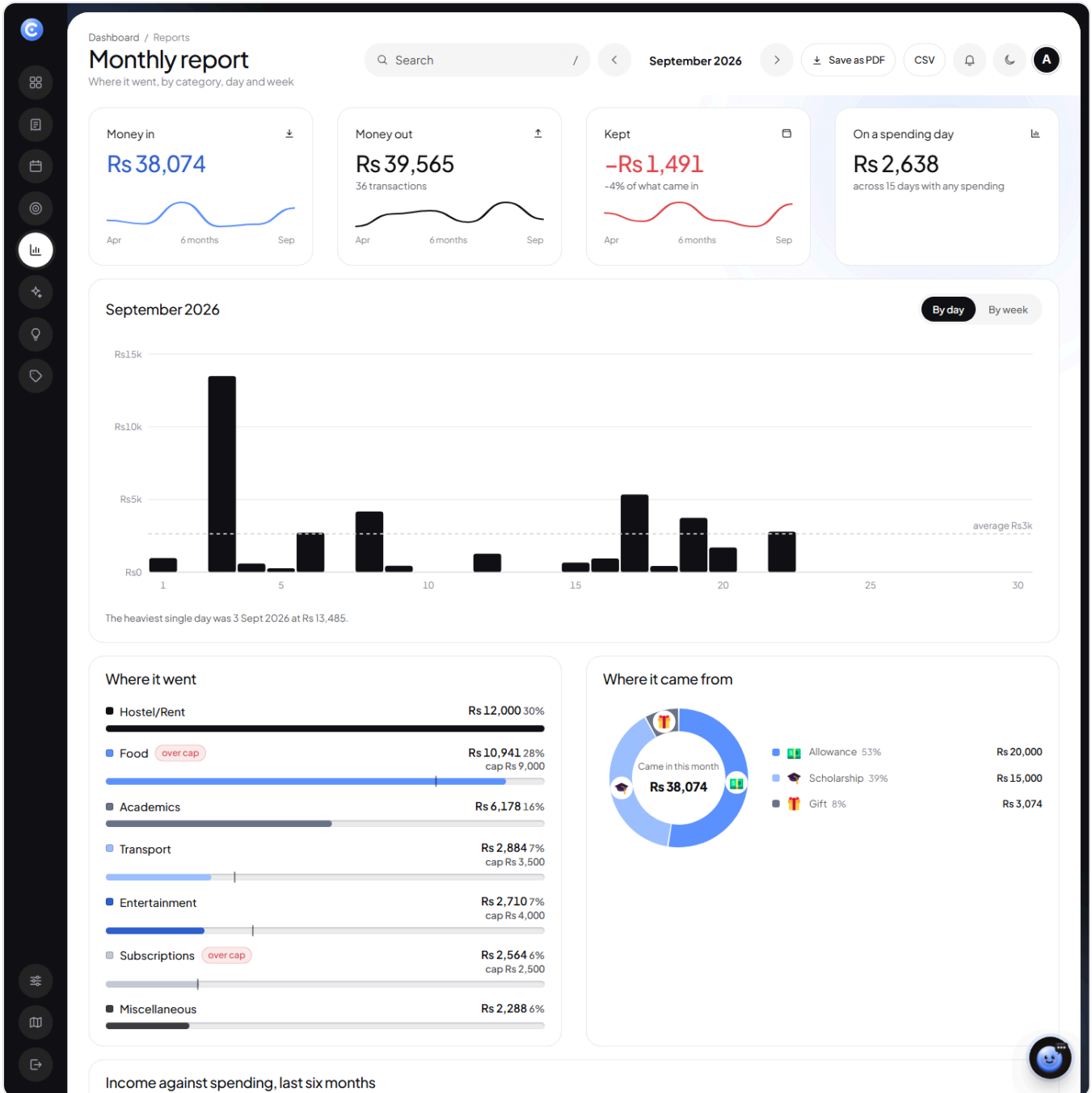
-Rs 428



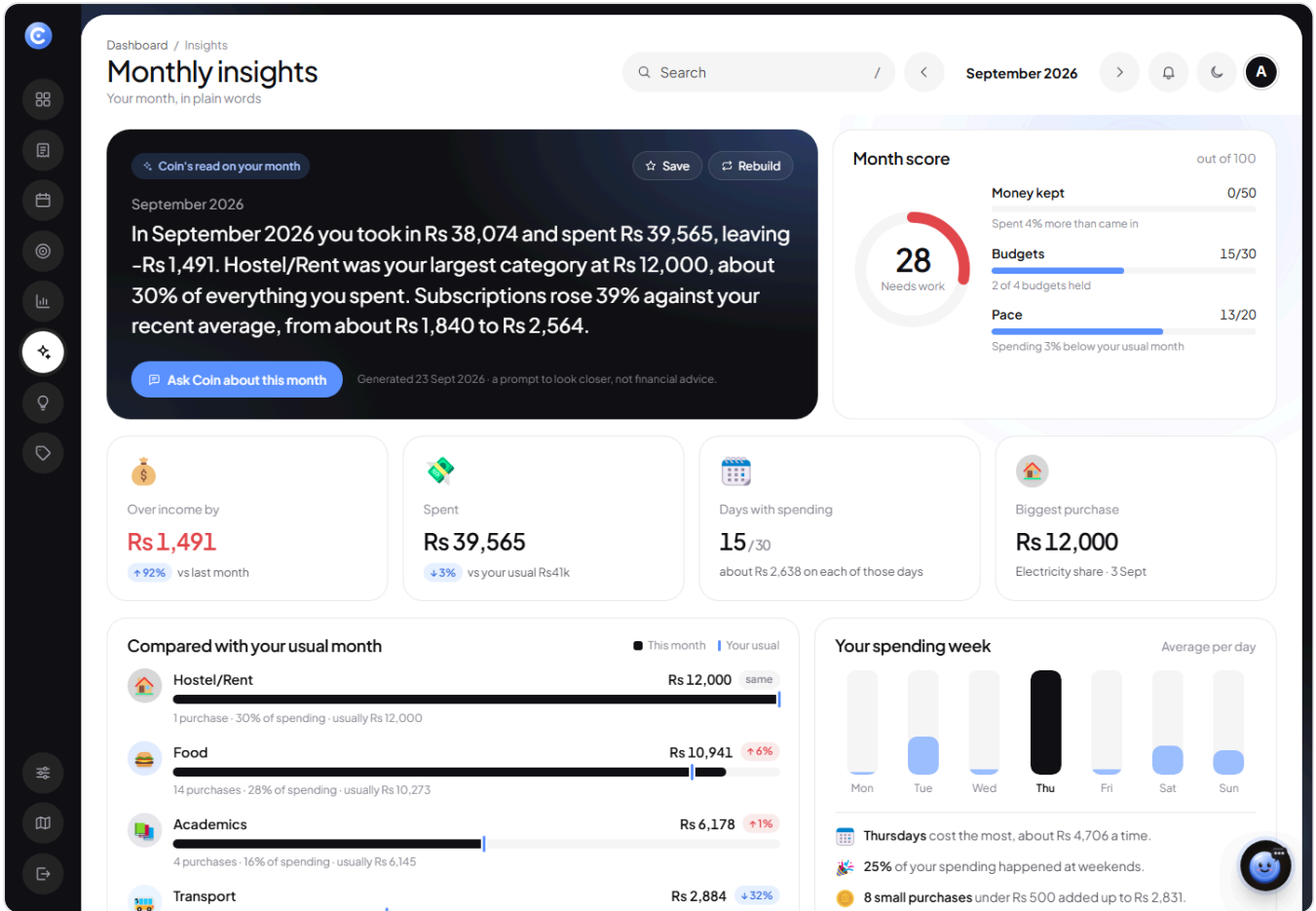
Money calendar: each day's spending and the selected day's transactions



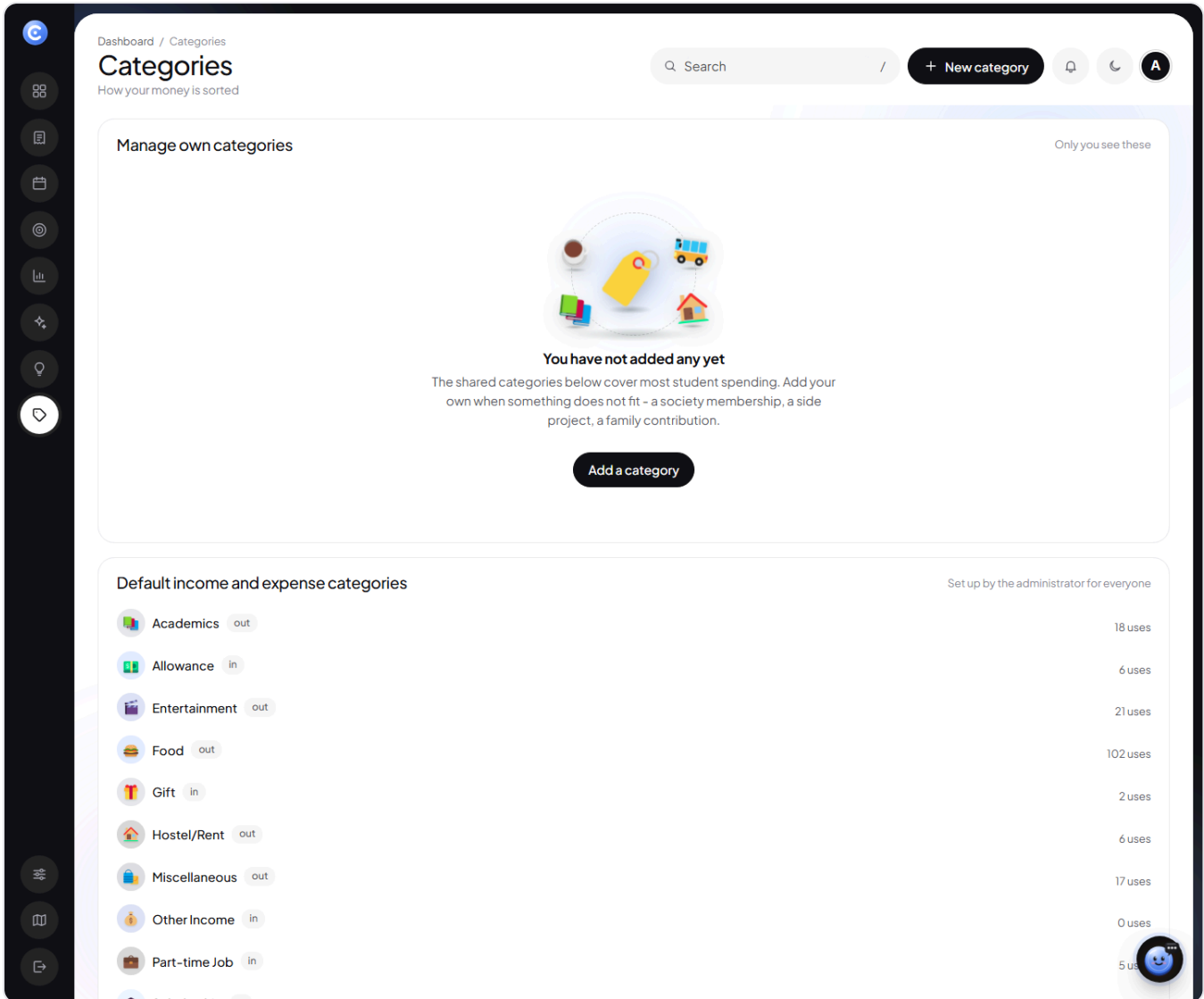
Budgets with live progress bars



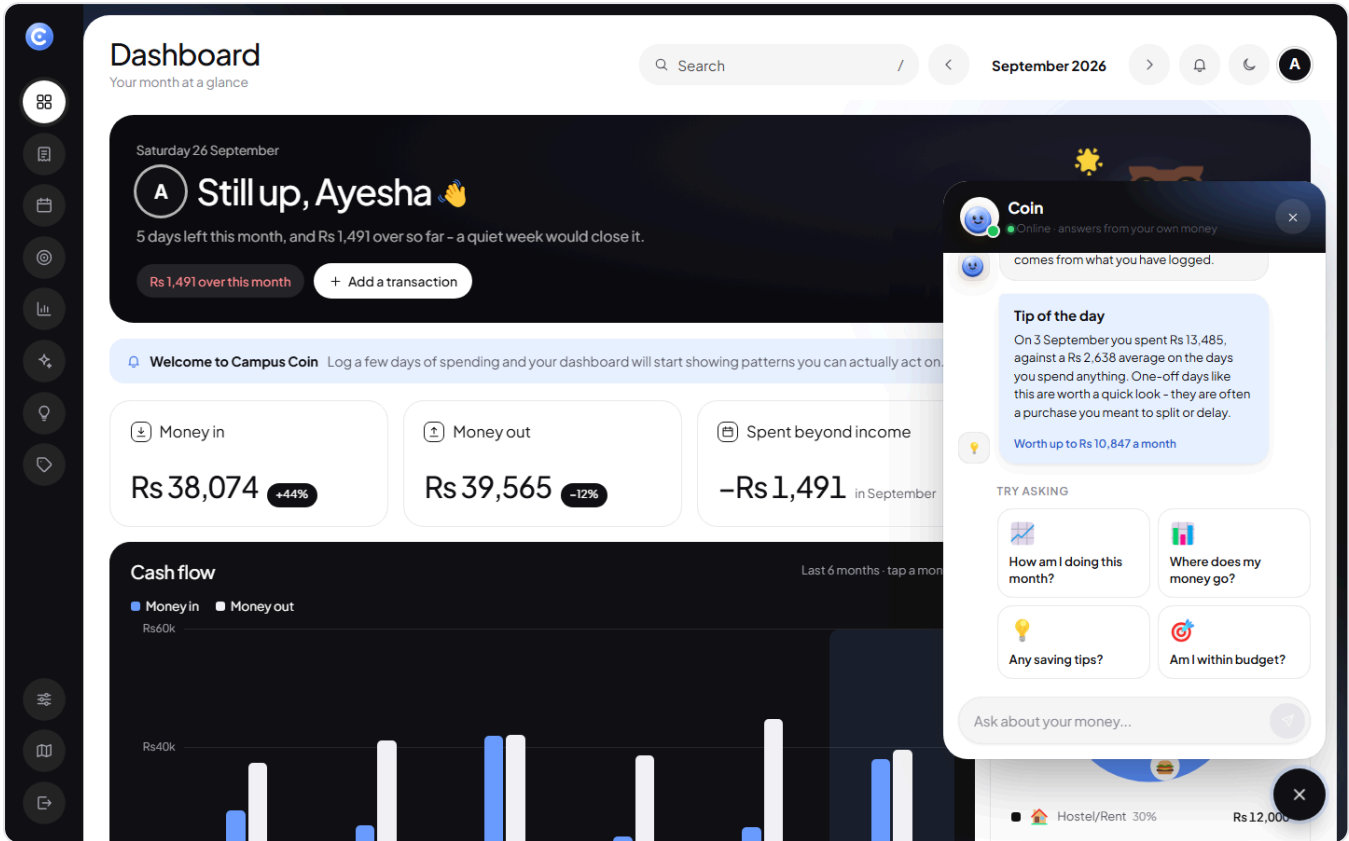
Monthly report



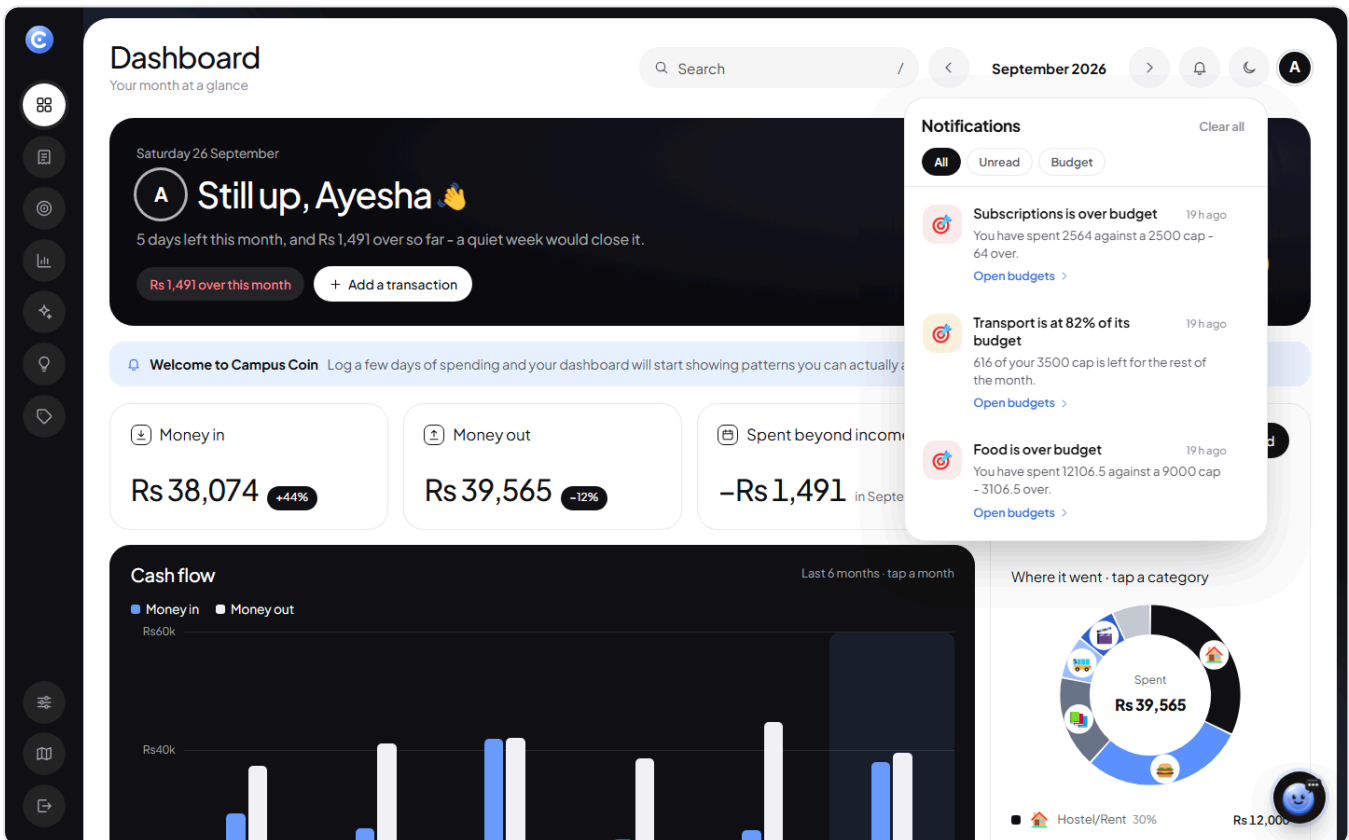
Monthly insights



Categories: manage own categories and the defaults



Coin, the chat assistant



Notifications panel

 Coin

[⬆ Upload photo](#)

Ayesha Khan

student@campuscoin.app

Your email is how you sign in and cannot be changed here.

Currency

PKR

Aptech Metro Star Gate

Monthly savings goal

20000

5000

Your savings goal is what the tips measure against, so it is worth setting honestly.

Save profile

Match device

e

Everything on the page scales together, including the charts.

☒ Tell me when a budget is close or broken

Set when you registered

🔒 This is the shared demo account, so its password stays as published. Register your own account to try changing a password.

For your safety, a password only changes through a link sent to student@campuscoin.app. Open it within the hour to choose a new one.

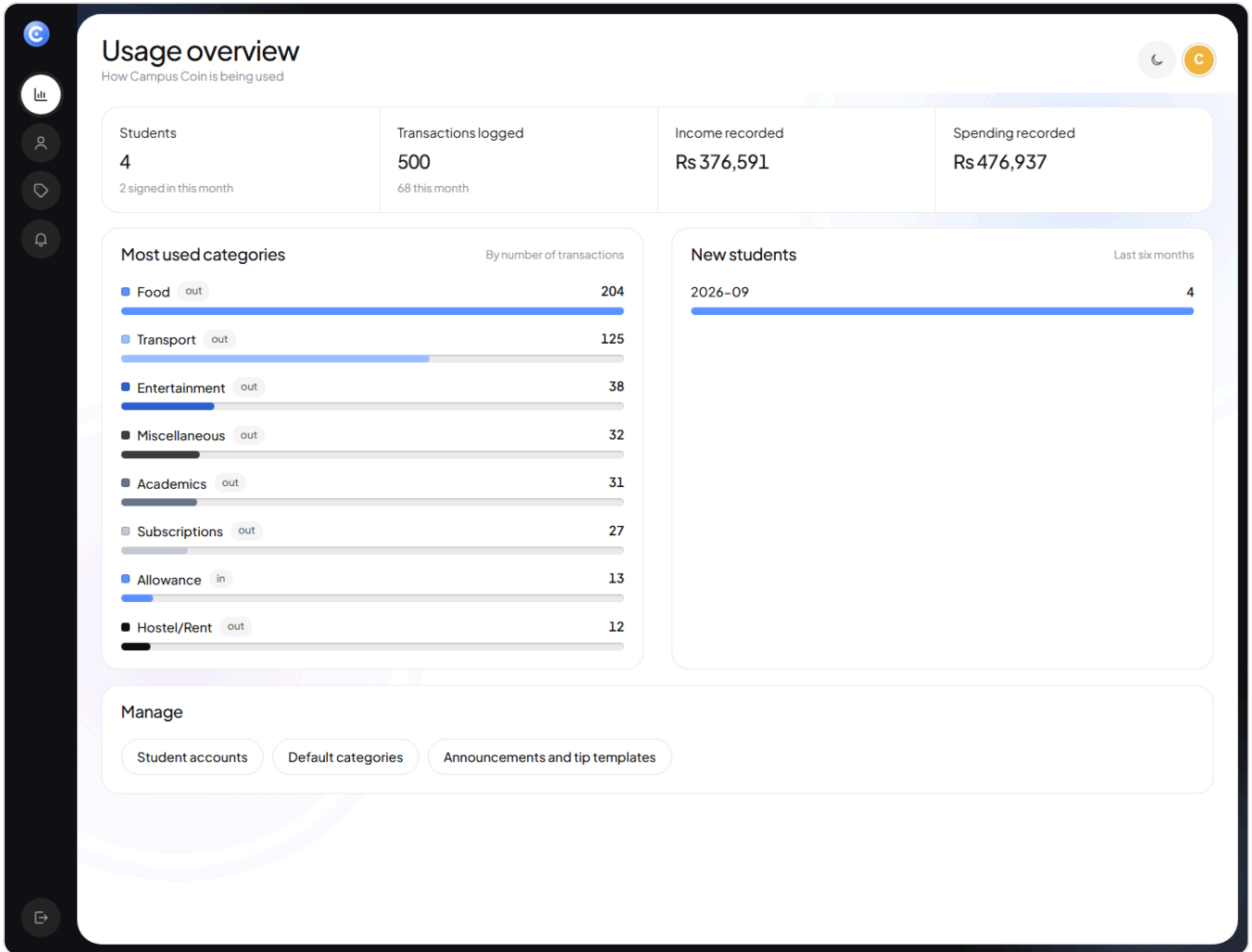
 [Email me a link](#)

Ends every other session - a lab computer, an old phone. This device stays signed in.

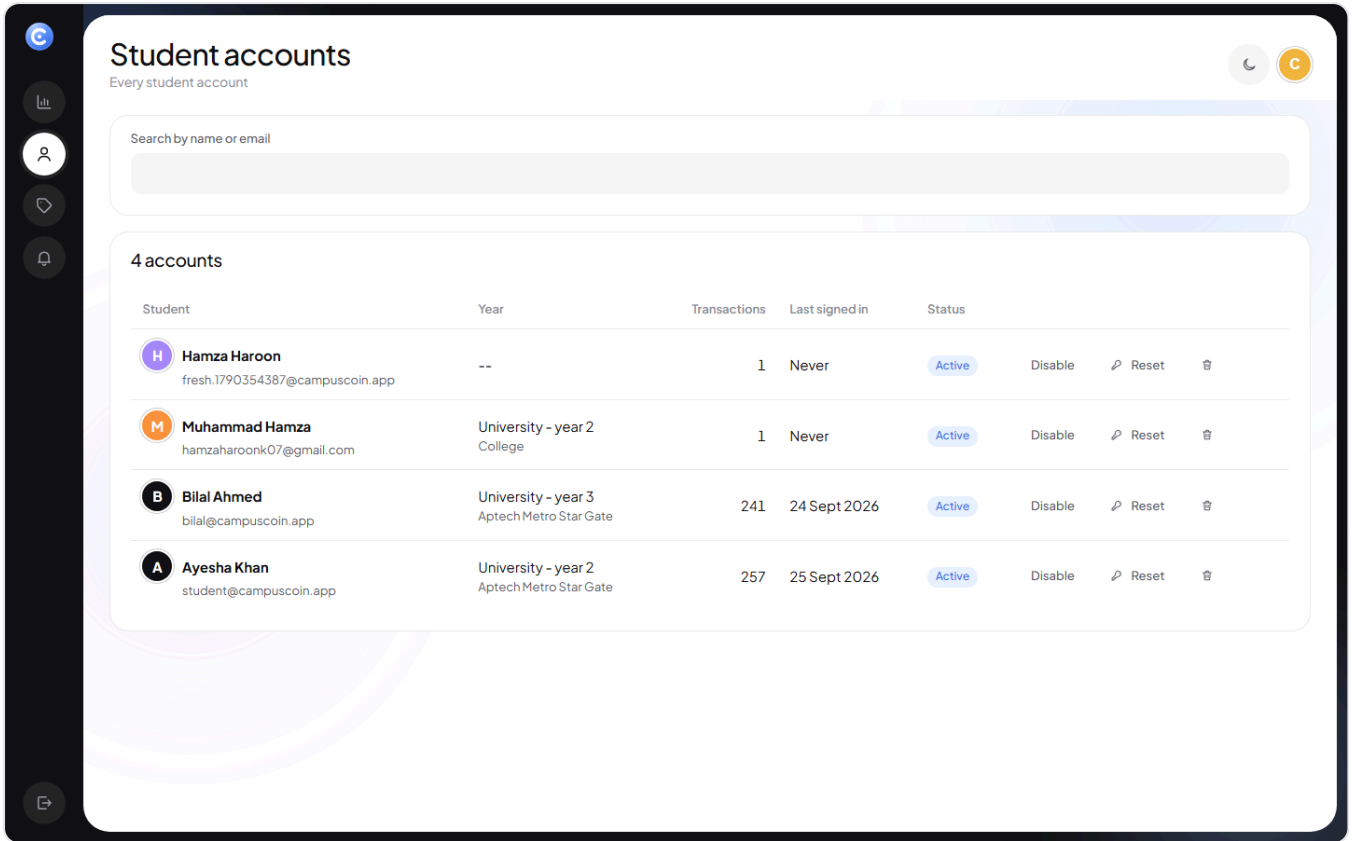
[Sign out everywhere](#)

- ✔ Passwords are stored only as salted bcrypt hashes. Five wrong tries lock sign-in for 15 minutes.

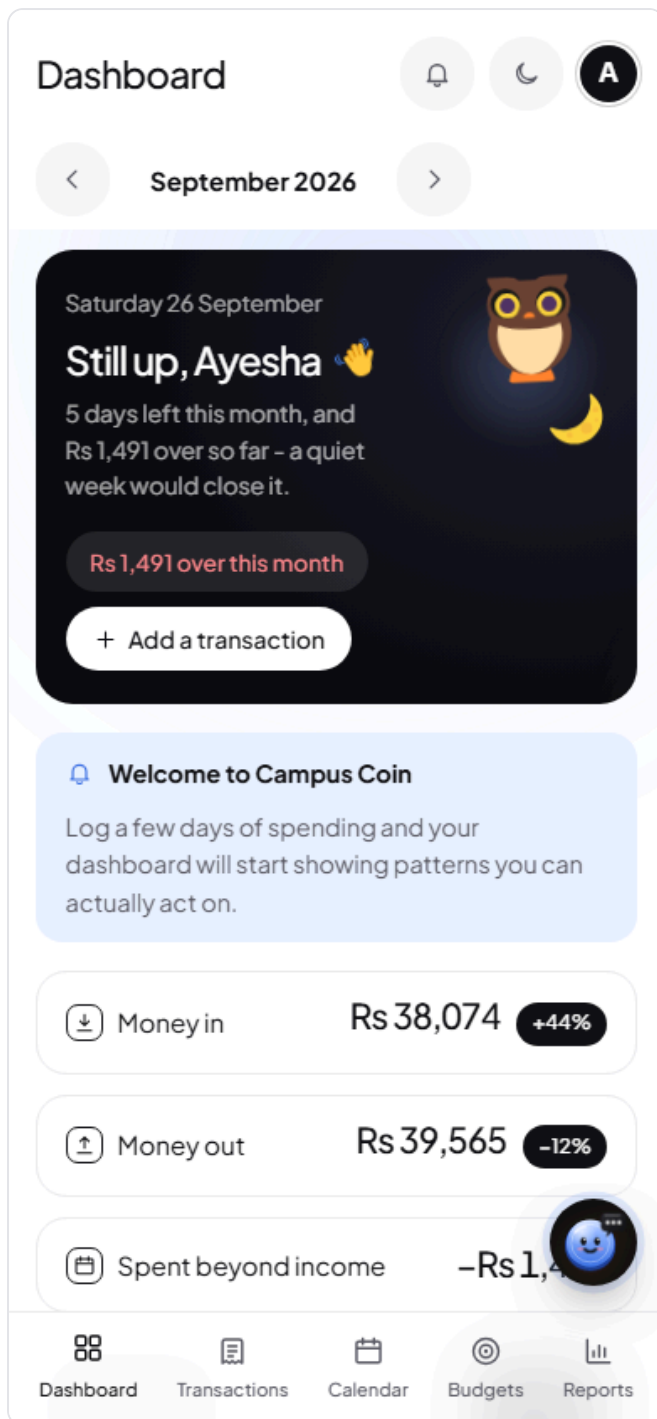
Every description you save teaches Coin which words mean which category. Correcting a suggestion is how it improves. None of this is shared, and suggestions are never financial advice.



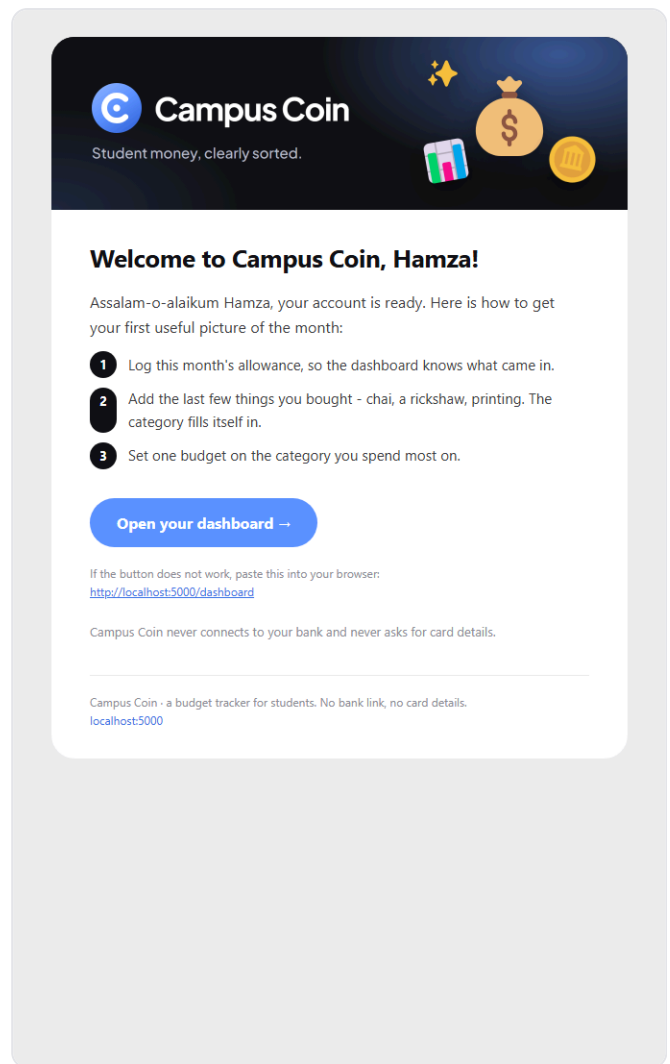
Admin control panel: usage statistics



Admin: student accounts



Dashboard on a phone



Welcome email

9. Test Data and Testing

9.1 Test data

The first start of the application seeds the database automatically, so every screen can be tested with realistic data straight away:

- **12 default categories:** Allowance, Part-time Job, Scholarship, Gift, Other Income; Food, Transport, Hostel/Rent, Academics, Subscriptions, Entertainment, Miscellaneous – each with keywords for the categoriser.
- **Two demo students with six months of history** (about 250 transactions each), generated with a fixed random seed so the data is the same on every install.
- **Demo budgets** for the current month: Food Rs 9,000, Subscriptions Rs 2,500, Transport Rs 3,500 and Entertainment Rs 4,000 – two of them deliberately exceeded so alerts can be seen.
- **Recurring entries** such as the monthly allowance and Spotify Premium, and a welcome announcement.
- **Two sample receipts** (a café bill and a bookshop bill) in `client/public/samples` for testing the receipt scanner.

Category	Seeded pattern per month	Example descriptions
Allowance	Rs 20,000 on the 2nd	Monthly allowance from home
Part-time Job	Rs 3,000 – 9,000, most months	Tutoring payment, Weekend shift, Freelance design work
Scholarship	Rs 15,000 every third month	Merit scholarship instalment
Gift	Rs 1,000 – 5,000, occasionally	Birthday money, Eidi from uncle
Food	14 – 22 purchases, Rs 180 – 900	Canteen chai and paratha, Foodpanda dinner, Biryani with friends
Transport	8 – 16 trips, Rs 120 – 600	Rickshaw to campus, Careem to the station, InDrive home
Hostel/Rent	Rs 12,000 once	Monthly hostel rent, PTCL wifi bill
Academics	1 – 4 purchases, Rs 400 – 3,500	Printing lab report, Data structures textbook
Subscriptions	2 – 3, Rs 500 – 1,500	Spotify Premium, Netflix share, ChatGPT Plus share
Entertainment	2 – 5, Rs 600 – 2,500	Cinema ticket, Cricket match with the hostel
Miscellaneous	2 – 5, Rs 200 – 1,800	Haircut, Laundry, Medicine from the pharmacy

Sample CSV import file

```
date,description,amount,type
2026-09-01,Monthly allowance,20000,income
2026-09-03,Chai at the canteen,80,expense
2026-09-04,Rickshaw to campus,150,expense
2026-09-05,Printing notes,120,expense
2026-09-06,Foodpanda dinner,650,expense
```

9.2 Test cases

An automated end-to-end test (`server/test/e2e.mjs`) registers a fresh student, works through every functional requirement against the running API, then deletes the account. It is run against the local server and the live site; the latest run passed all 65 checks on both.

#	Area	Test case	Result
1	Auth	Register a student with profile fields	Pass
2	Auth	Duplicate email is refused	Pass
3	Auth	Log in	Pass
4	Auth	Wrong password is refused	Pass
5	Auth	Admin sign-in refuses a student	Pass
6	Auth	Protected routes need a session	Pass
7	Auth	Edit profile (name, year, allowance, goal)	Pass
8	Auth	Year of study covers school, college and university	Pass
9	Auth	Theme and font size saved to the account	Pass
10	Auth	Weak passwords are refused	Pass
11	Auth	Password cannot change without an emailed link	Pass
12	Auth	Change password by link ends other sessions	Pass
13	Auth	Sign out everywhere	Pass
14	Auth	The shared demo password cannot be changed	Pass
15	Auth	Password recovery by tokenised link	Pass
16	Categories	Default income and expense categories exist	Pass
17	Categories	Create a personal category	Pass
18	Categories	Edit a personal category	Pass
19	Categories	Delete a personal category	Pass
20	Categories	Default categories cannot be edited by a student	Pass
21	Transactions	Quick-add income	Pass
22	Transactions	Quick-add expenses	Pass
23	Transactions	Invalid amount is refused	Pass
24	Transactions	Recurring entry (monthly subscription)	Pass
25	Transactions	Edit a transaction	Pass
26	Transactions	Delete a transaction	Pass
27	Transactions	List, search and filter	Pass
28	Transactions	Recently viewed/edited list	Pass
29	Anomalies	Unusually large expense is flagged	Pass
30	Anomalies	Duplicate transaction is flagged	Pass

#	Area	Test case	Result
31	AI assistant	Suggests a category as you type	Pass
32	AI assistant	Learns from a correction	Pass
33	AI assistant	Override is recorded (accuracy tracked)	Pass
34	Import	CSV preview with batch category suggestions	Pass
35	Import	CSV commit	Pass
36	Import	CSV export	Pass
37	Budgets	Set a monthly budget per category	Pass
38	Budgets	Real-time progress (spent vs cap)	Pass
39	Budgets	In-app notification when a category nears/exceeds budget	Pass
40	Budgets	Copy budgets to next month	Pass
41	Dashboard	Greeting data, balance, top category, budget vs actual, tips	Pass
42	Reports	Category-wise monthly report	Pass
43	Reports	Calendar groups the month by day	Pass
44	Reports	Daily and weekly summaries	Pass
45	Reports	Income vs expense, last six months	Pass
46	Reports	Filter by date range, category and income source	Pass
47	Reports	Forecast needs 3 months (honest refusal for a new account)	Pass
48	Reports	Flagged transactions list	Pass
49	Insights	Generate the monthly narrative insight	Pass
50	Insights	Insight history is stored	Pass
51	Insights	Bookmark an insight	Pass
52	Insights	Share the summary by email	Pass
53	Tips	Tips generated from own history, ranked by impact	Pass
54	Tips	Pin and dismiss a tip	Pass
55	Chat	Answers questions from own data	Pass
56	Admin	Separate admin login	Pass
57	Admin	Students cannot reach admin routes	Pass
58	Admin	Usage statistics (active users, transactions, top categories)	Pass
59	Admin	View and search users	Pass
60	Admin	Disable a user blocks login, enable restores it	Pass
61	Admin	Reset a user password	Pass
62	Admin	Repeated wrong passwords lock the account	Pass
63	Admin	Add, edit and remove a default category	Pass
64	Admin	Announcements and tip templates	Pass

#	Area	Test case	Result
65	Admin	Remove a user account (clean-up)	Pass

9.3 Manual testing

Scenario	Steps	Expected result
Quick add	Dashboard → Chai tile → enter 80 → Save	Saved under Food; dashboard totals update
AI suggestion	Add transaction → type "Careem to uni"	Transport suggested with its reason; can be changed
Receipt scan	Add transaction → Scan receipt → sample café bill	Amount, shop and date filled in
Budget alert	Set Food budget below current spending	Notification "Food is over budget" appears in the bell
Password link	Settings → Email me a link → open email → set password	Signed in with the new password; other devices signed out
Lockout	Enter a wrong password five times	Sign-in refused for 15 minutes
Phone layout	Open on a 390px-wide screen	Bottom tab bar, stacked cards, no sideways scrolling

10. Project Installation Instructions

10.1 Requirements

- Node.js 18 or newer (includes npm) - <https://nodejs.org>
- A MongoDB server is optional: with none configured, the application starts its own embedded MongoDB in `server/data/db`.
- Any current browser (Chrome, Edge, Firefox, Safari).

10.2 Steps

```
# 1. Get the code (or unzip the submitted folder)
git clone https://github.com/hamzaharoonk07-cloud/campuscoin.git
cd campuscoin

# 2. Install the server and client packages
npm run setup

# 3. Create the settings file (every value in it is optional)
copy server\.env.example server\.env          (Windows)
cp server/.env.example server/.env            (macOS / Linux)

# 4. Build the React front end
npm run build

# 5. Start the application
npm start
```

Open **<http://localhost:5000>** in a browser. On the first start the database is seeded with the default categories, the administrator and the two demo students, so every screen has data immediately.

10.3 Development mode

```
npm run dev          # API on http://localhost:5000
npm run dev:client   # React on http://localhost:5173 (hot reload)
```

10.4 Optional settings (server/.env)

Variable	Purpose
<code>MONGO_URI</code>	Use an external MongoDB (local or Atlas) instead of the embedded one
<code>JWT_SECRET</code>	Secret used to sign session tokens

Variable	Purpose
SMTP_HOST , SMTP_PORT , SMTP_USER , SMTP_PASS , MAIL_FROM	Send real emails (for Gmail: smtp.gmail.com, 465, the address and an app password). Without them, reset links are shown on screen on a local machine.
ANTHROPIC_API_KEY	Let Claude reword monthly summaries; without it the built-in writer is used
CRON_SECRET	Protect the daily recurring-transactions job on Vercel

10.5 Running the tests

```
npm start           # in one terminal
npm run test:e2e --prefix server  # in a second terminal
```

10.6 Live deployment

The same code is deployed on Vercel with a MongoDB Atlas database: <https://campuscoin-sable.vercel.app>. Every push to the `main` branch of the GitHub repository deploys automatically.

11. User Credentials

These accounts are created automatically on first start and exist on the live site. Their passwords are fixed so that evaluators can always sign in; they cannot be changed, reset or locked.

Administrator

Sign in at `/admin/login`

Email: `admin@campuscoin.app`

Password: `Admin@12345`

Student (demo)

Sign in at `/login`

Email:

`student@campuscoin.app`

Password: `Student@12345`

Ayesha Khan, University year
2

Student (second)

Sign in at `/login`

Email: `bilal@campuscoin.app`

Password: `Student@12345`

Bilal Ahmed, University year
3

User type	What they can do
Student	Record, import and scan transactions; manage own categories, budgets and profile; view dashboard, calendar, reports, insights and tips; chat with Coin; receive alerts and emails
Administrator	View usage statistics; view, disable, reset or remove student accounts; manage default categories; publish announcements and tip templates

New student accounts can be created from the Register page at any time. A student account cannot open the admin panel even with the admin URL.

12. Acknowledgement of AI Tools

As required by the SRS, the AI tools used during this project are acknowledged here.

Tool	How it was used
Claude Code (Anthropic)	Coding assistant used during development for writing and debugging code, design iterations, screenshots and testing, and help in drafting this report. All code and text were reviewed, and the design decisions are understood and can be explained.
Anthropic Claude API (optional feature)	Inside the application, when an API key is configured, it rewrites already-computed monthly figures in friendlier words and answers chat questions the built-in rules do not recognise. It never calculates any figure.
Tesseract.js	Open-source OCR engine used in the browser to read receipt photos.

Other resources: Microsoft Fluent Emoji (MIT licence) for category pictures, Simple Icons (CC0) for merchant logos, and design inspiration from finance dashboards on Dribbble. No ready-made website template was used.

13. Conclusion and Future Work

Campus Coin meets the functional and non-functional requirements of the SRS. A student can record money in seconds, see where it goes, set budgets that warn in time, and read advice that is built from their own history rather than general rules. The application is deployed publicly, works on phones and computers, and is covered by an automated test that exercises every requirement.

Possible future work

- Shared budgets for hostel roommates splitting rent and groceries.
- Reading SMS or email payment alerts from JazzCash and Easypaisa to add transactions automatically.
- An Urdu interface option.
- Savings goals for a target date (a laptop, a trip) with progress tracking.
- Installable progressive web app with offline entry.